ЛР1.Індивідуальний робочий процес програміста із використанням методології Scrum, контролера робочого часу Clockify, системи управління проектами Trello та системи керування версіями GIT

1. Git Configuration

Перевірка встановленої версії Git та налаштування глобальних параметрів користувача (ім'я користувача та email):

```
ivankrlk@Ivans-MacBook-Pro ~ % git --version
git version 2.55.0
ivankrlk@Ivans-MacBook-Pro ~ % git config --global user.name "fall6nluv"
ivankrlk@Ivans-MacBook-Pro ~ % git config --global user.email "mc000kref@gmail.com"
```

```
git --version

git config --global user.name "fall6nluv"

git config --global user.email "mc000kref@gmail.com"

```

2. Creating Project Directory

Створення нової робочої директорії та перехід до неї:

```
ivankrlk@Ivans-MacBook-Pro Documents % mkdir Git_Lab1
ivankrlk@Ivans-MacBook-Pro Documents % cd Git_Lab1
ivankrlk@Ivans-MacBook-Pro Git_Lab1 %
```

```
mkdir Git_Lab1

cd Git_Lab1

```

					ЛР.ОК13.ПІ241.12.01		
Змін.	Аркуш	№ докум.	Підпис	Дата			
Розробив		Кирилюк І.І.			Trello, Clockify, git	Літ: У	Аркуш: 1 / Аркушів: 52
Перевірив		Щуцький С. В.					
Н.контр.						ХПК	
Затвер.							

3. Initializing Git Repository

Ініціалізація порожнього репозиторію Git у поточній папці:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/ivankrlk/Documents/Git_Lab1/.git/
```

`git init`

4. Making Commits & Viewing History

Створення файлів, перевірка статусу, індексування, комміт змін та перегляд журналу коммітів:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "Hello Git" > test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.txt

nothing added to commit but untracked files present (use "git add" to track)
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git add test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git commit -m "First commit"
[master (root-commit) 8d40825] First commit
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "<h1>Hello</h1>" > index.html
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git add .
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git commit -m "Second commit"
[master 59aef60] Second commit
 1 file changed, 1 insertion(+)
 create mode 100644 index.html
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git log --oneline
59aef60 (HEAD -> master) Second commit
8d40825 First commit
```

```
echo "Hello Git" > test.txt

git status

git add test.txt

git commit -m "First commit"

echo "<h1>Hello</h1>" > index.html

git add .

git commit -m "Second commit"

git log --oneline
```

5. Checking Out Commits, Discarding Changes & Reverting

Перехід на конкретний комміт (стан detached HEAD), повернення до гілки master, скасування локальних змін у файлі та скасування комміту за допомогою revert:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout 8d40825
Note: switching to '8d40825'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 8d40825 First commit
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout master
Previous HEAD position was 8d40825 First commit
Switched to branch 'master'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "Bad code" >> test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout test.txt
Updated 1 path from the index
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git revert HEAD --no-edit
[master 926d8e1] Revert "Second commit"
 Date: Fri Sep 11 13:50:06 2026 +0300
 1 file changed, 1 deletion(-)
 delete mode 100644 index.html
```

```
git checkout 8d40825

git checkout master

echo "Bad code" >> test.txt

git checkout test.txt

git revert HEAD --no-edit
```

6. Creating & Working in a Feature Branch

Перегляд списку гілок, створення та перехід до нової гілки розробки, а також комміт змін у неї:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git branch
* master
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout -b dev
Switched to a new branch 'dev'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "Dev branch changes" > dev.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git add dev.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git commit -m "Added dev.txt"
[dev bcc4381] Added dev.txt
 1 file changed, 1 insertion(+)
 create mode 100644 dev.txt
```

```
git branch

git checkout -b dev

echo "Dev branch changes" > dev.txt

git add dev.txt

git commit -m "Added dev.txt"
```

7. Merging & Rebasing Branches

Виконання швидкого злиття (fast-forward merge) гілки *dev* у *master*, створення гілки *feature* зі змінами, внесення паралельних коммітів у *master* та перенесення (rebase) гілки *feature* на *master*:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout master
Switched to branch 'master'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git merge dev
Updating 926d8e1..bcc4381
Fast-forward
 dev.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 dev.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout master
Already on 'master'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout -b feature
Switched to a new branch 'feature'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "important work lmao" > feature.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git add feature.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git commit -m "Commit on feature branch"
[feature a530c43] Commit on feature branch
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout master
Switched to branch 'master'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "new thing in master" >> test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git add test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git commit -m "New commit on master"
[master 0de039f] New commit on master
 1 file changed, 1 insertion(+)
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git checkout feature
Switched to branch 'feature'
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git rebase master
Successfully rebased and updated refs/heads/feature.
```

```
git checkout master

git merge dev

git checkout -b feature

echo "important work lmao" > feature.txt

git add feature.txt

git commit -m "Commit on feature branch"

git checkout master

echo "new thing in master" >> test.txt

git add test.txt

git commit -m "New commit on master"

git checkout feature
```

```
git rebase master
```

8. Connecting to Remote & Initial Push

Підключення віддаленого репозиторію GitHub, перейменування основної гілки на main та відправка коммітів із налаштуванням відстеження:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git remote add origin https://github.com/Fallenluv-boo/OKR1_Kyryliuk.git
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git branch -M main
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git push -u origin main
Username for 'https://github.com': Fallenluv-boo
Password for 'https://Fallenluv-boo@github.com':
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 11 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (16/16), 1.38 KiB | 1.38 MiB/s, done.
Total 16 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Fallenluv-boo/OKR1_Kyryliuk.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

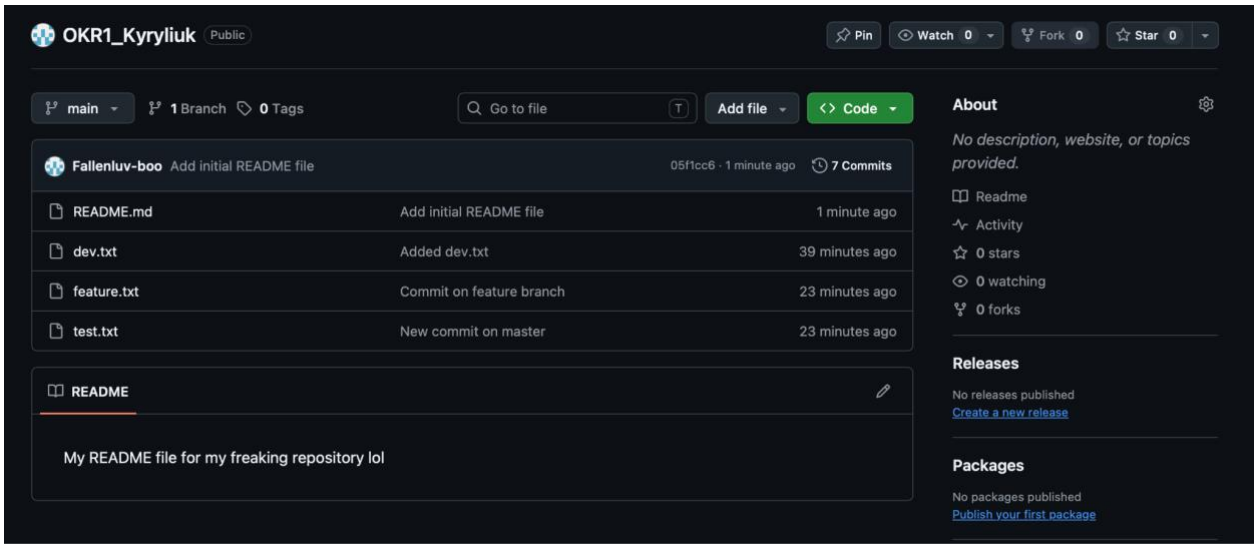
```
git remote add origin https://github.com/Fallenluv-boo/OKR1_Kyryliuk.git

git branch -M main

git push -u origin main
```

9. GitHub Repository Overview

Вигляд репозиторію на GitHub після успішної відправки початкових коммітів та структури гілки main.



10. Local Modifications Before Pulling

Внесення локальних змін та перевірка стану робочої директорії:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % echo "Some new local change" >> test.txt
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

```
echo "Some new local change" >> test.txt

git status
```

11. Stashing Local Changes & Pulling Remote Updates

Тимчасове збереження локальних змін (stash) для уникнення конфліктів під час pull, отримання оновлень з origin/main та відновлення збережених змін:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git pull --rebase origin main
error: cannot pull with rebase: You have unstaged changes.
error: Please commit or stash them.
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git stash
Saved working directory and index state WIP on main: 99c0230 Commit on feature branch
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git pull --rebase origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.00 KiB | 343.00 KiB/s, done.
From https://github.com/Fallenluv-boo/OKR1_Kyryliuk
 * branch            main       -> FETCH_HEAD
   99c0230..05f1cc6  main       -> origin/main
Updating 99c0230..05f1cc6
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git stash pop
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (f20af9a70097a6fd8ebad53905209216d416e459)
```

```
git pull --rebase origin main

git stash

git pull --rebase origin main

git stash pop
```

12. Committing Local Changes & Final Push

Індексування відновлених змін, створення локального комміту та відправка оновлень у віддалений репозиторій:

```
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git add .
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git commit -m "Local changes after pull"
[main 50560eb] Local changes after pull
 1 file changed, 1 insertion(+)
ivankrlk@Ivans-MacBook-Pro Git_Lab1 % git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 11 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 311 bytes | 311.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Fallenluv-boo/OKR1_Kyryliuk.git
   05f1cc6..50560eb  main -> main
```

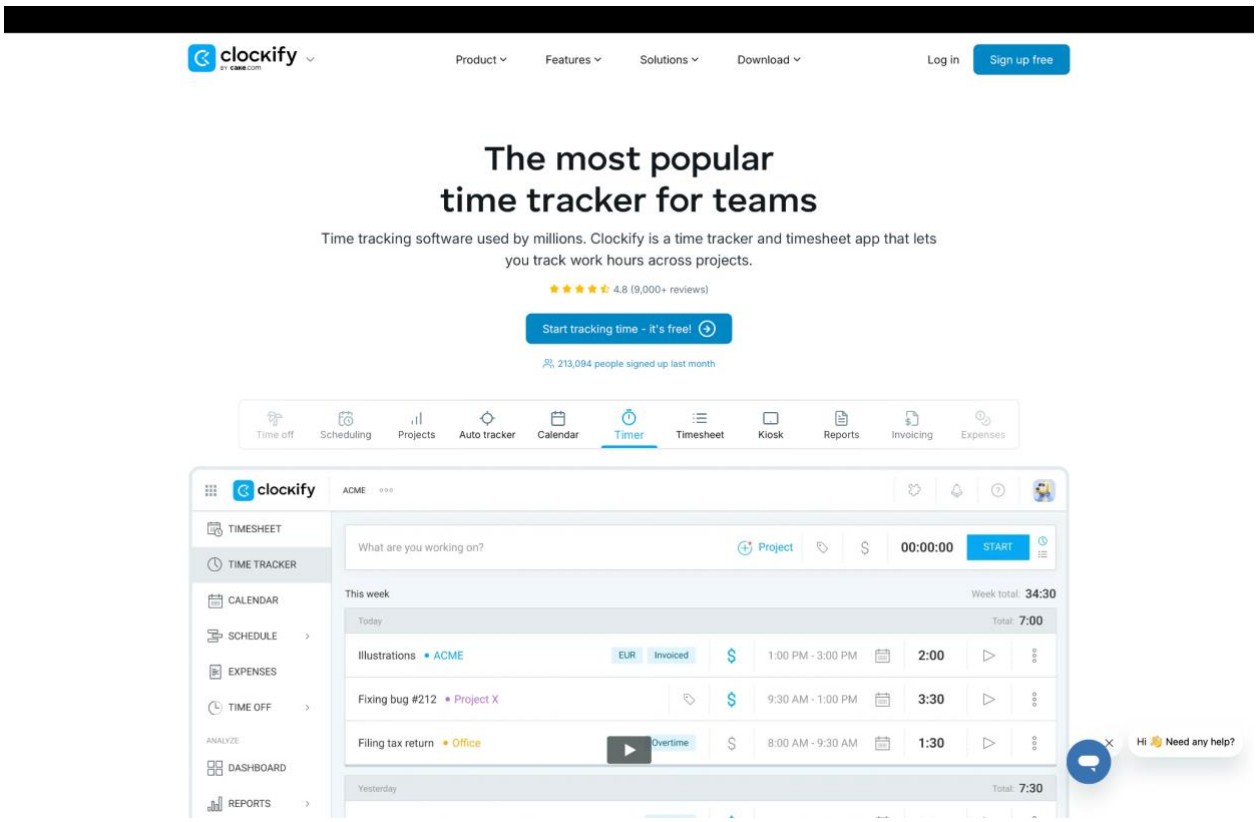
```
git add .

git commit -m "Local changes after pull"

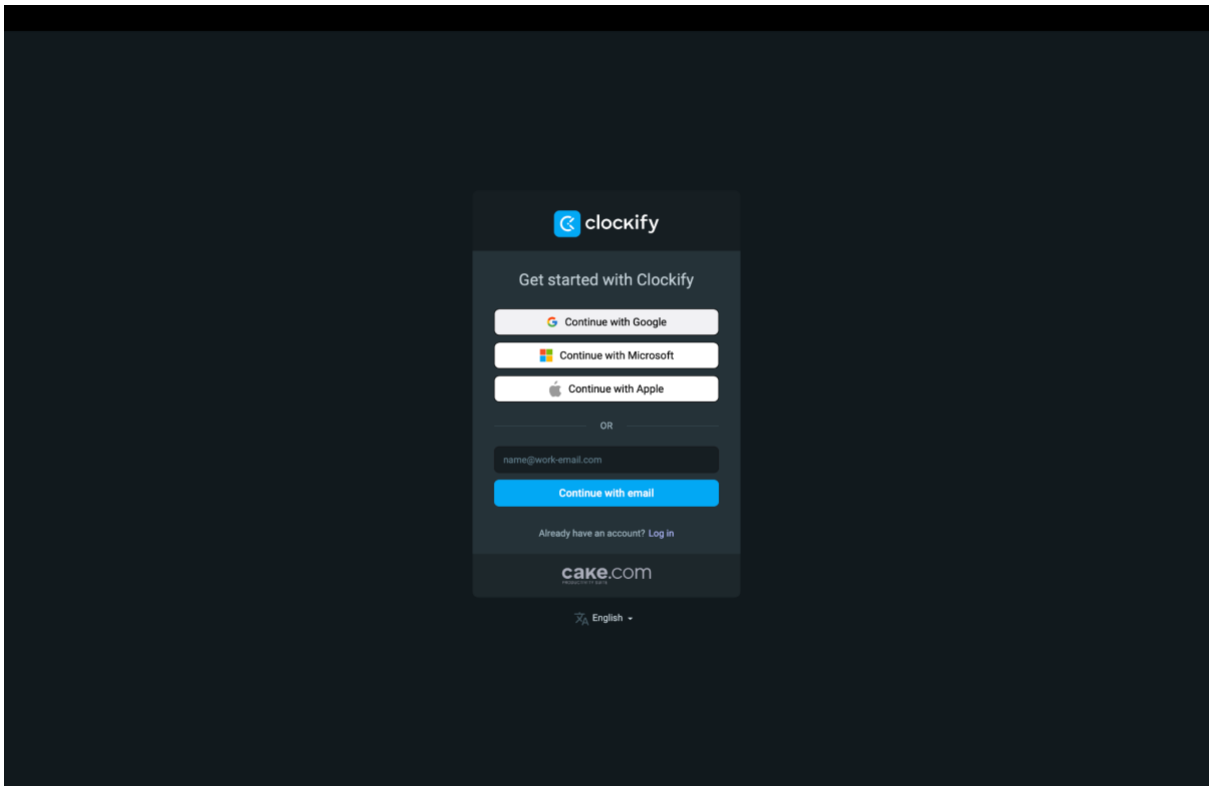
git push
```

Робота з Clockify

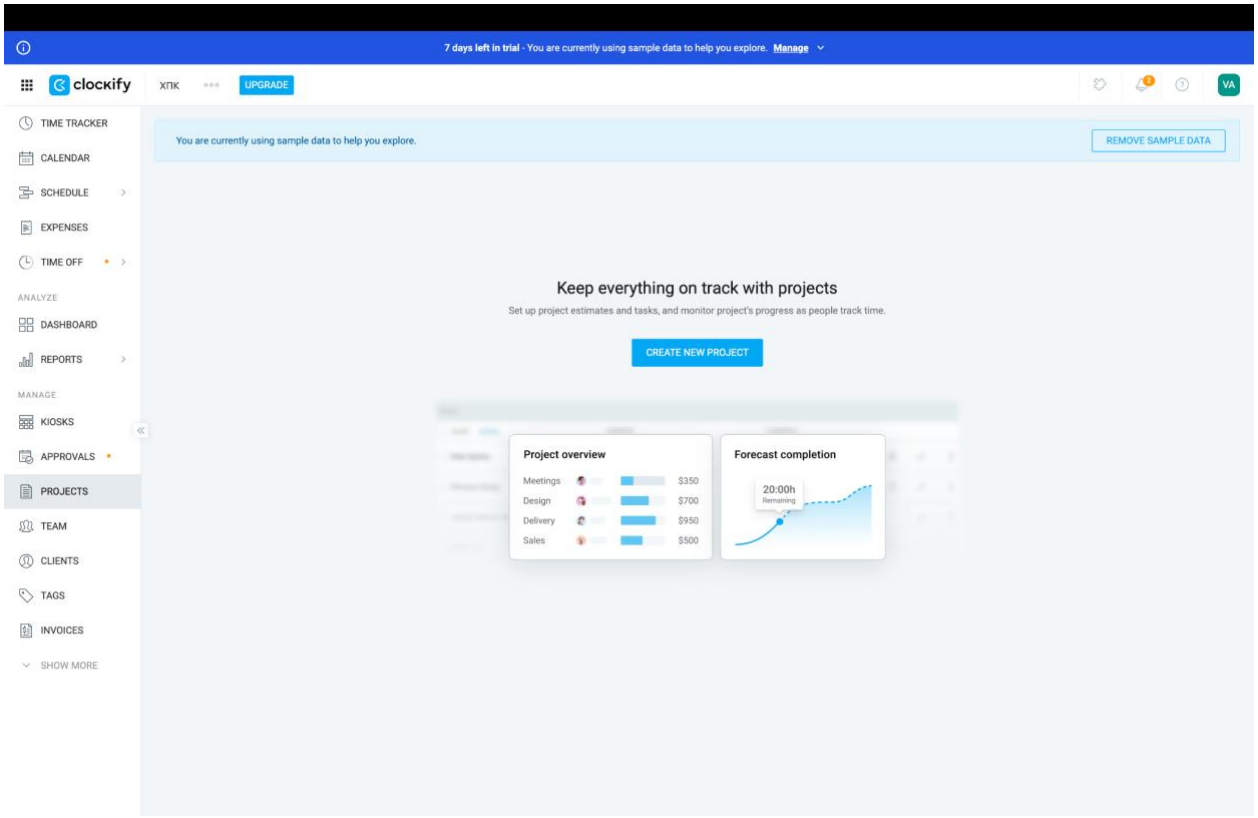
Головна сторінка сервісу Clockify для відстеження робочого часу:



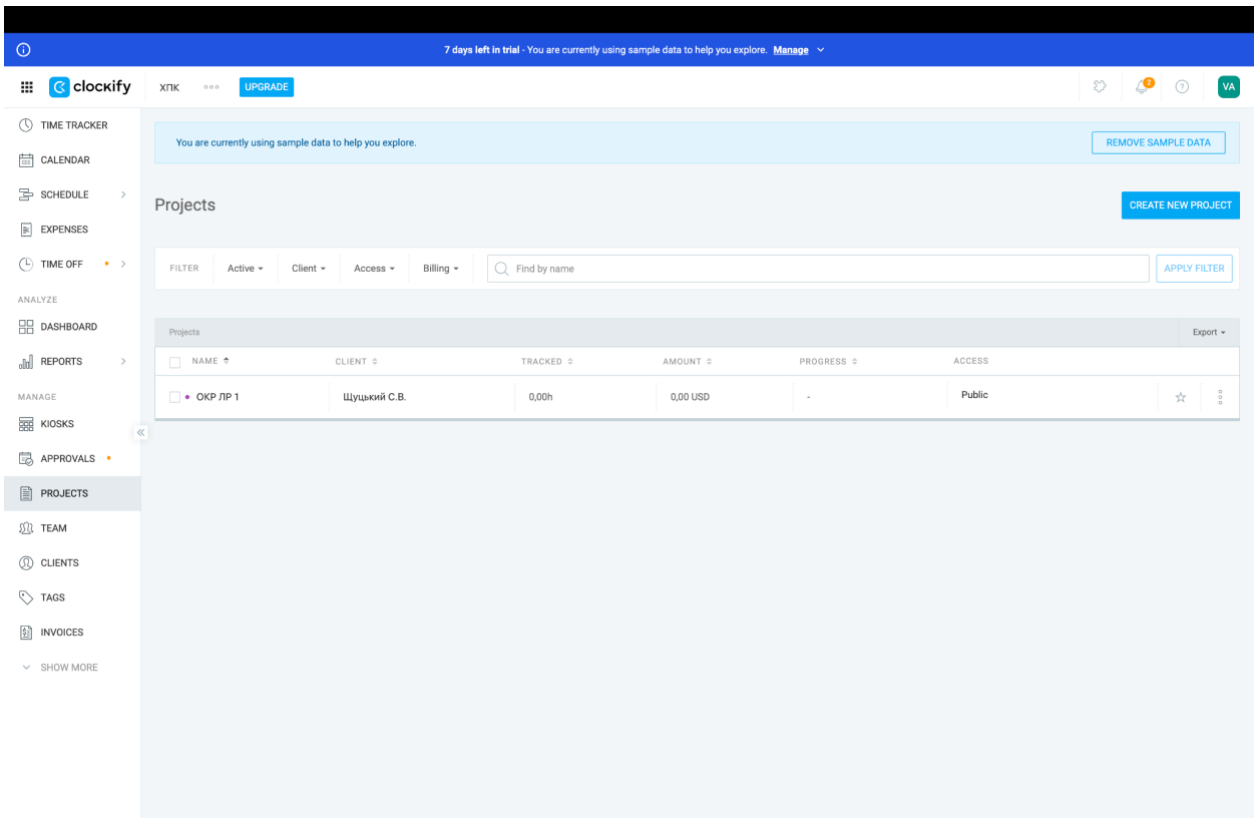
Форма авторизації та входу в обліковий запис Clockify:



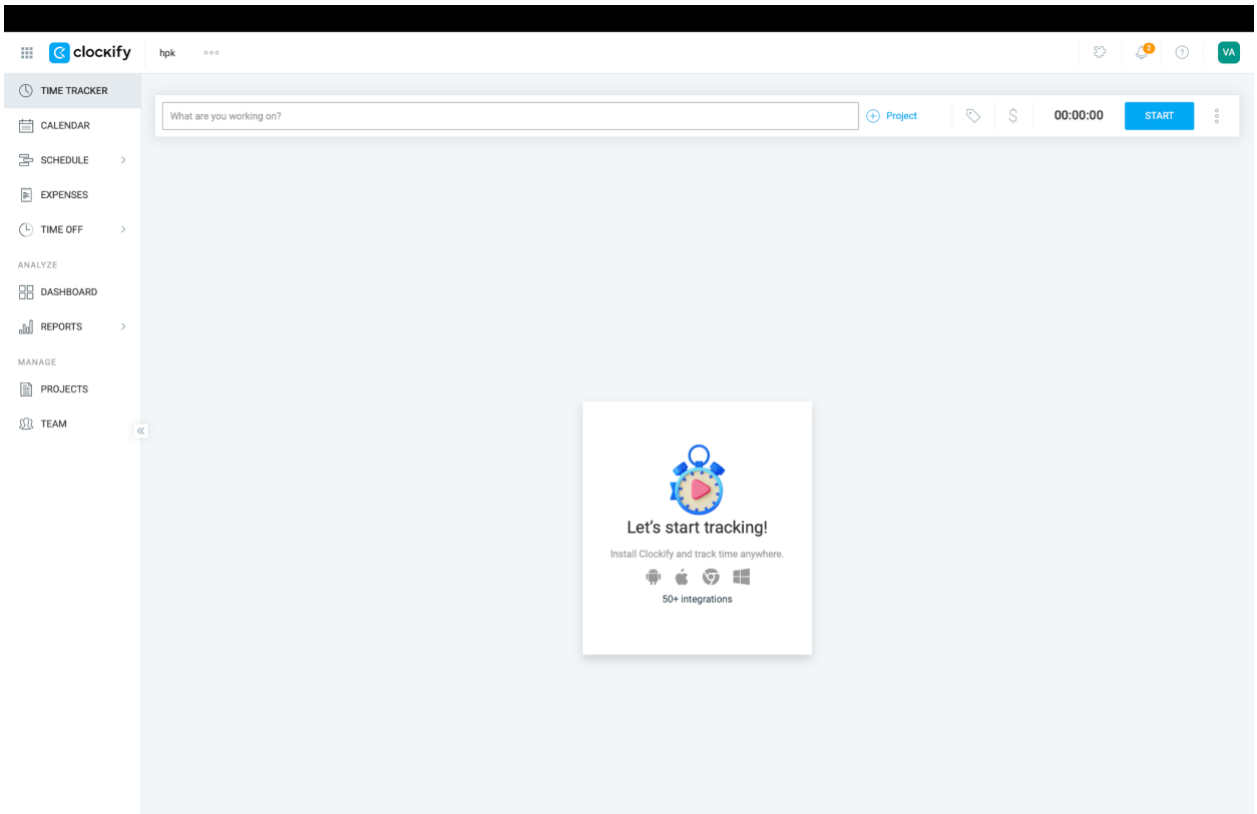
Розділ проєктів та створення нового проєкту:



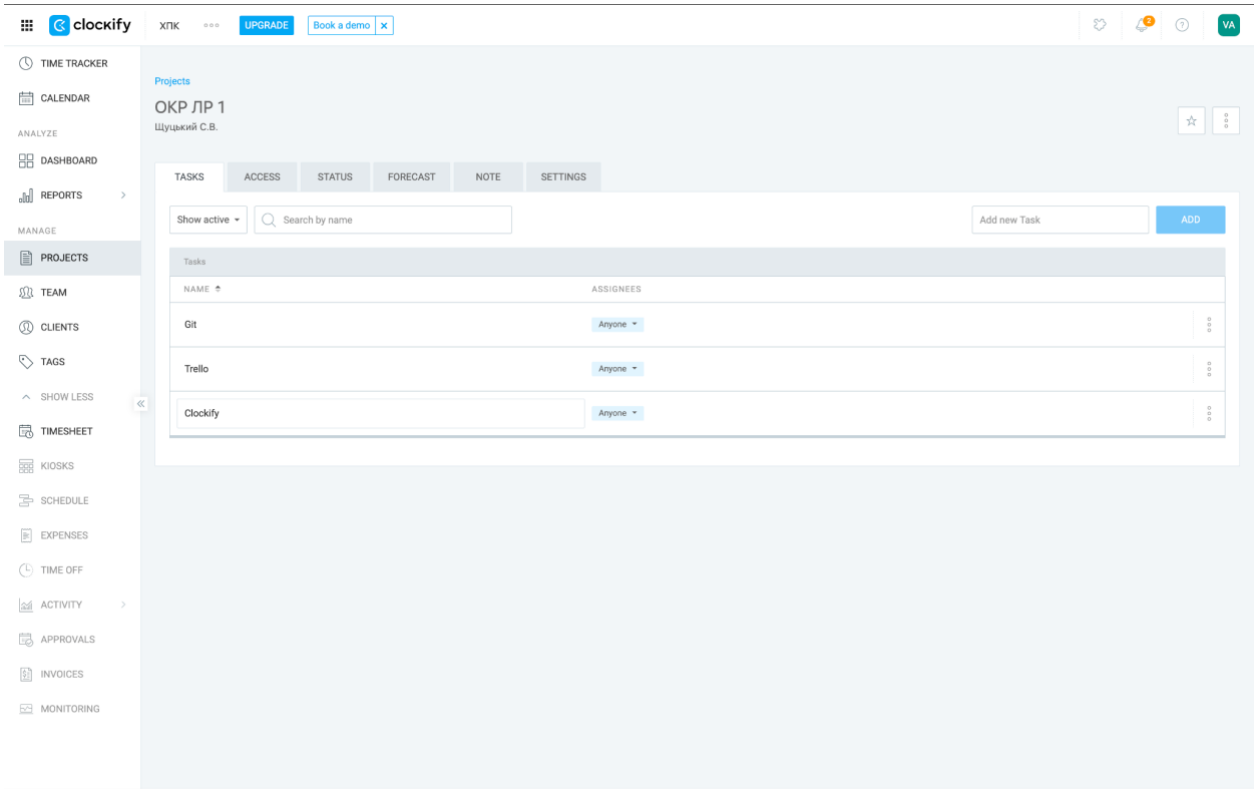
Перегляд списку активних проєктів у систему:



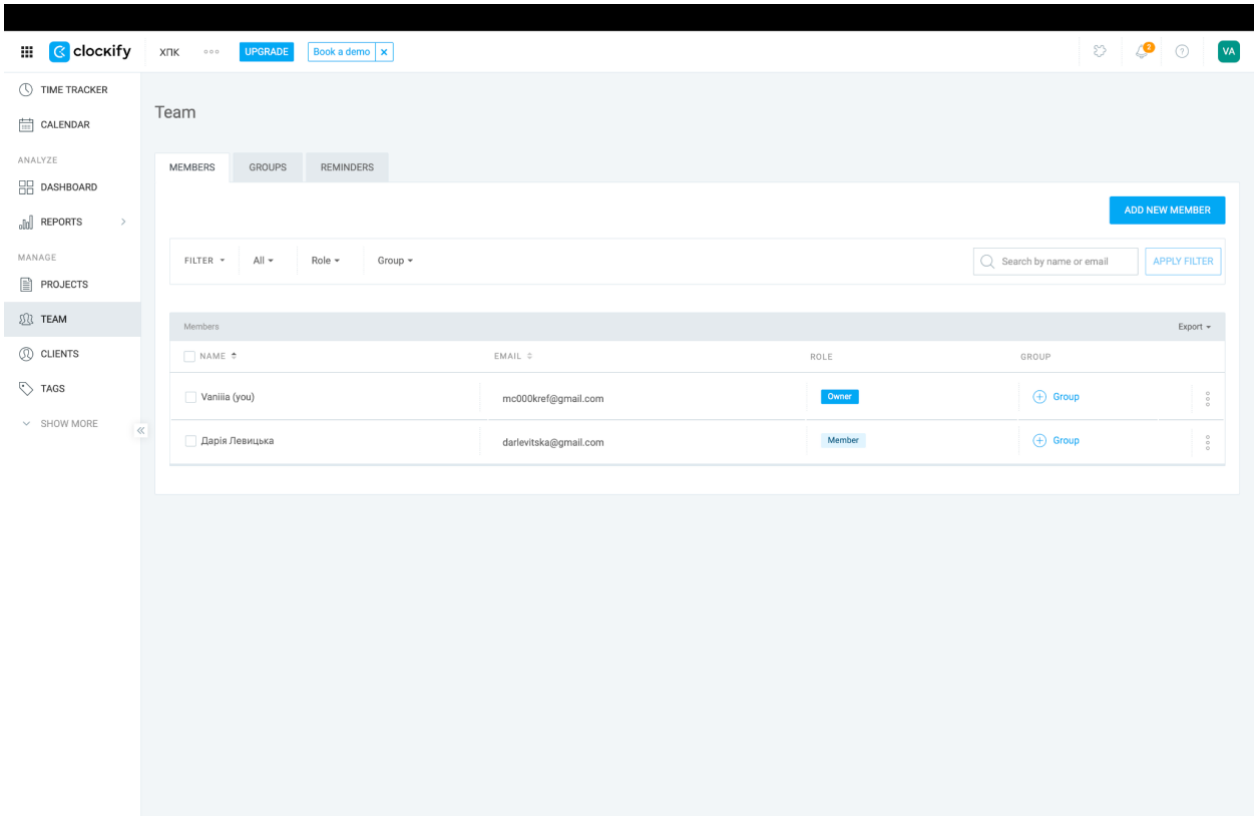
Початковий екран відстеження часу (Time Tracker):



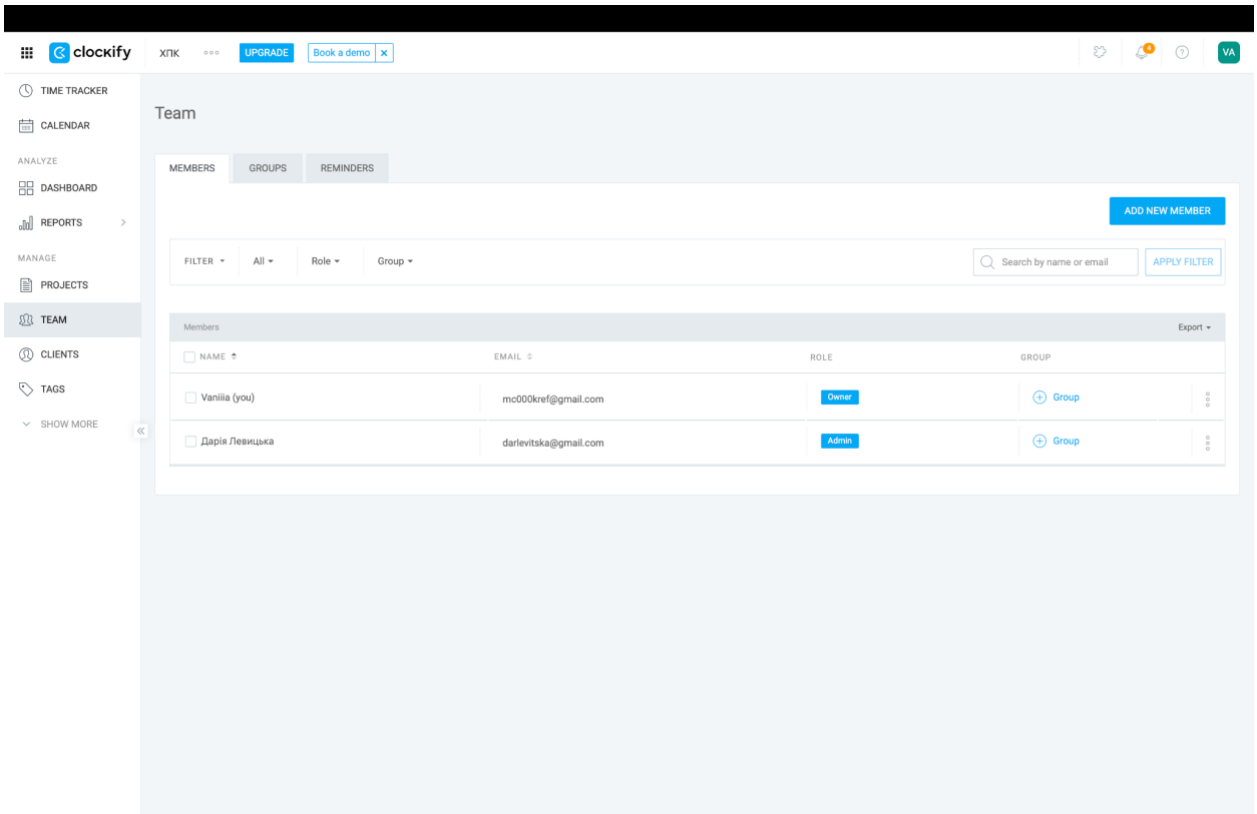
Додавання завдань у проєкті:



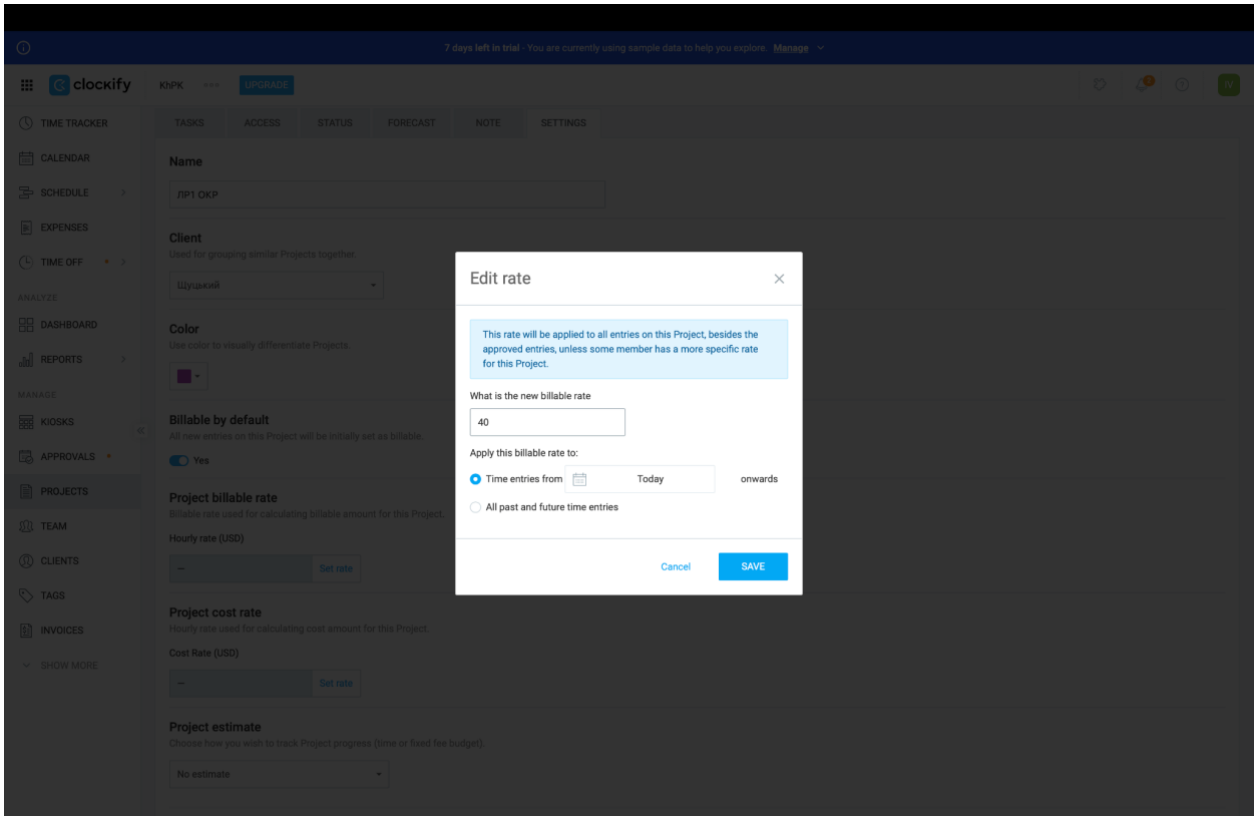
Управління командою та додавання нових учасників:



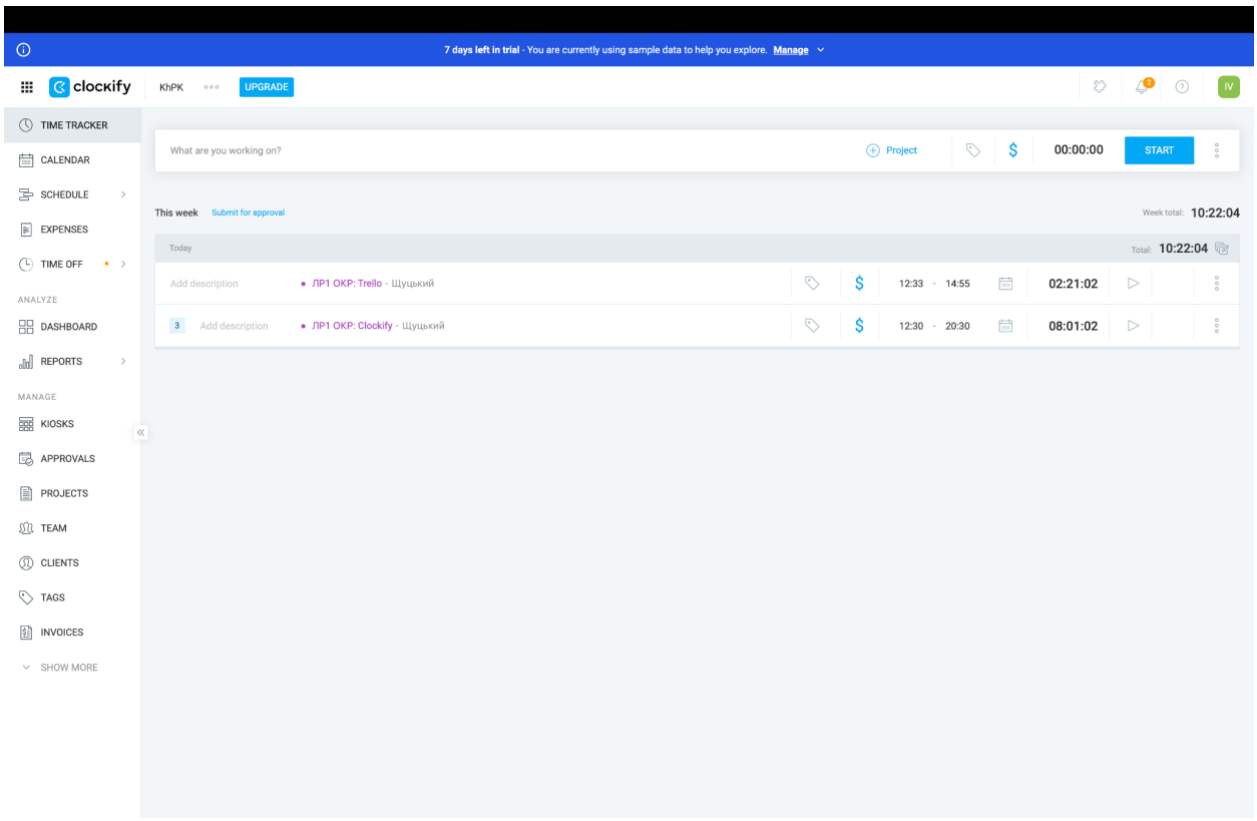
Зміна ролей та прав доступу учасників команди:



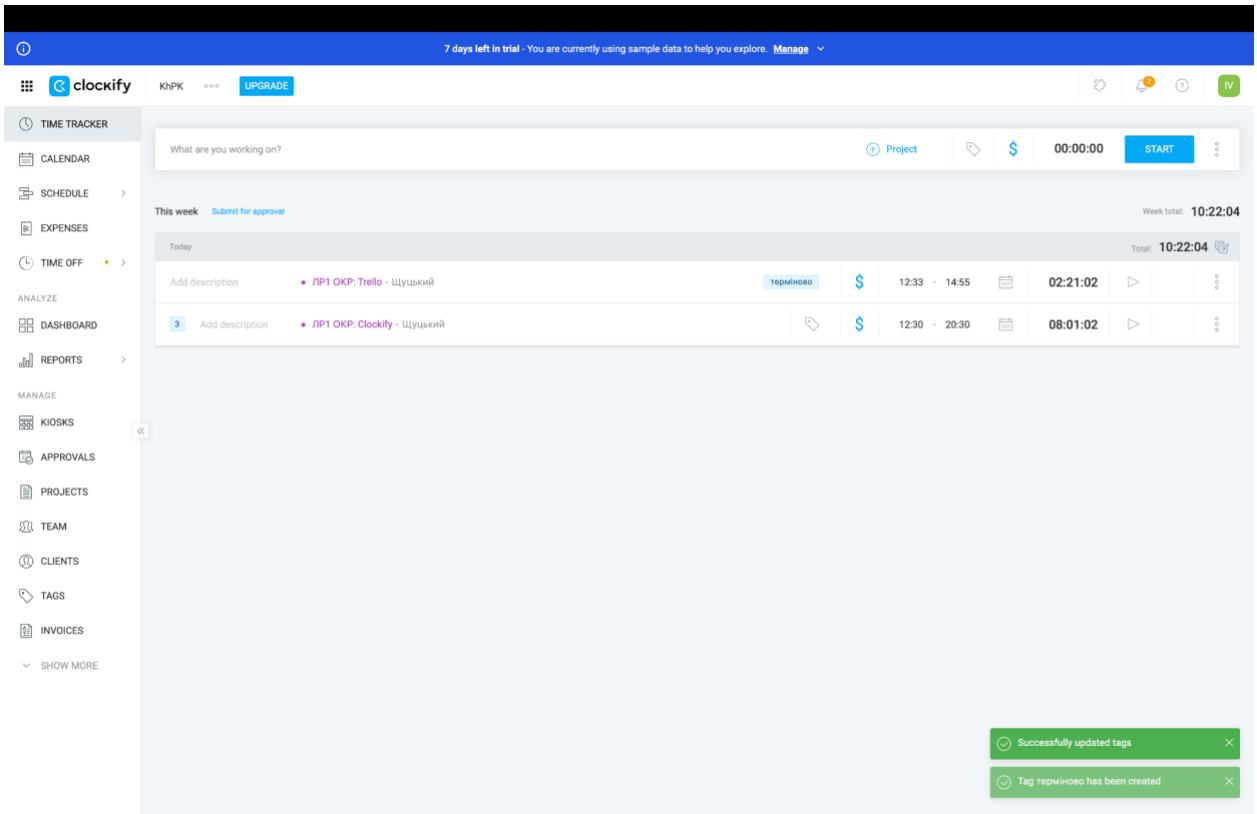
Налаштування погодинної ставки для проєкту:



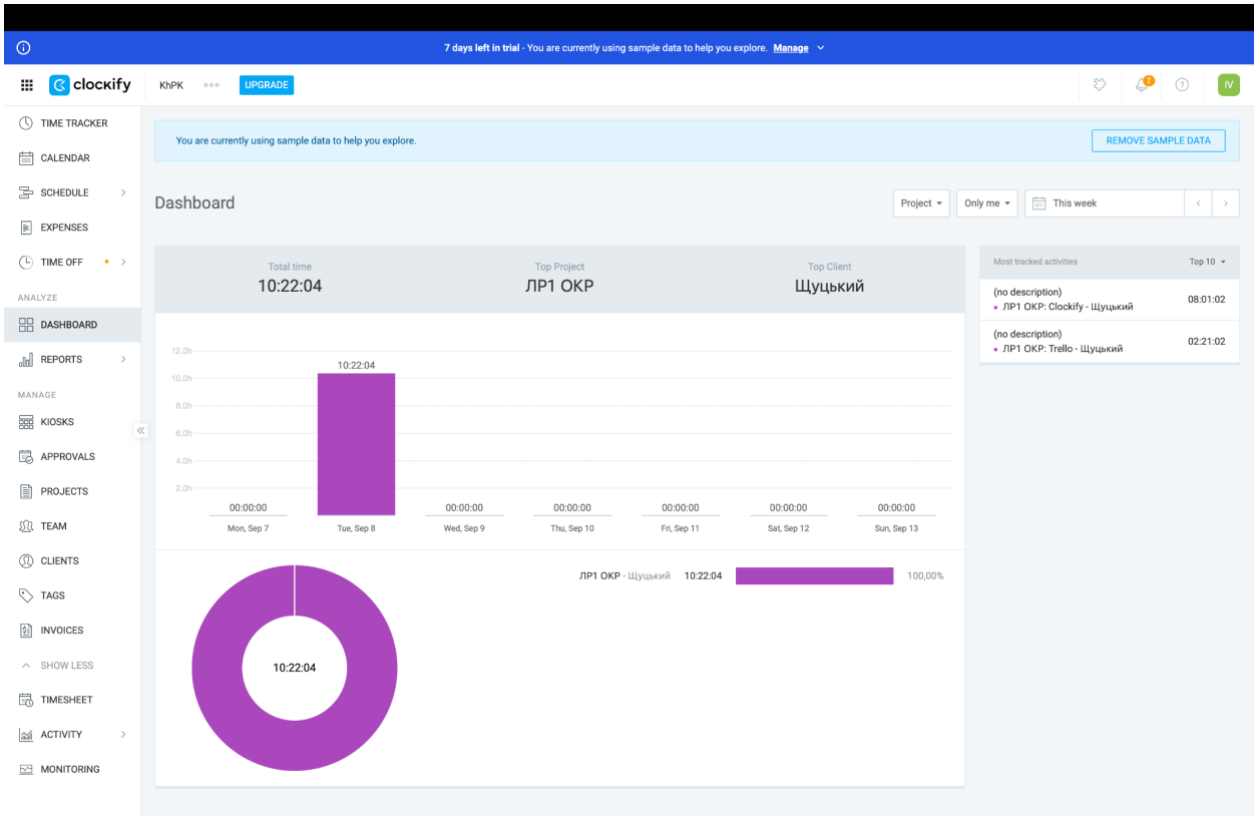
Запис відстеженого часу по завданнях:



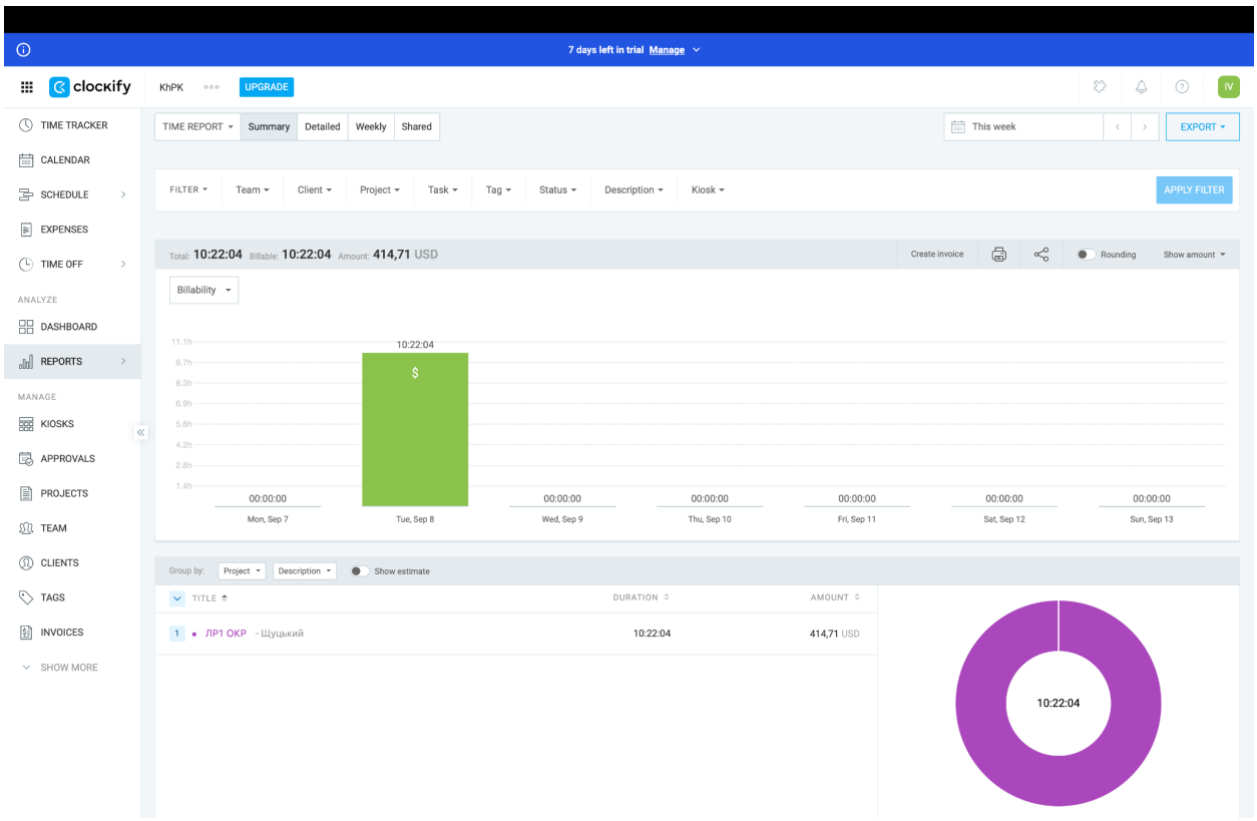
Додавання тегів до часових записів:



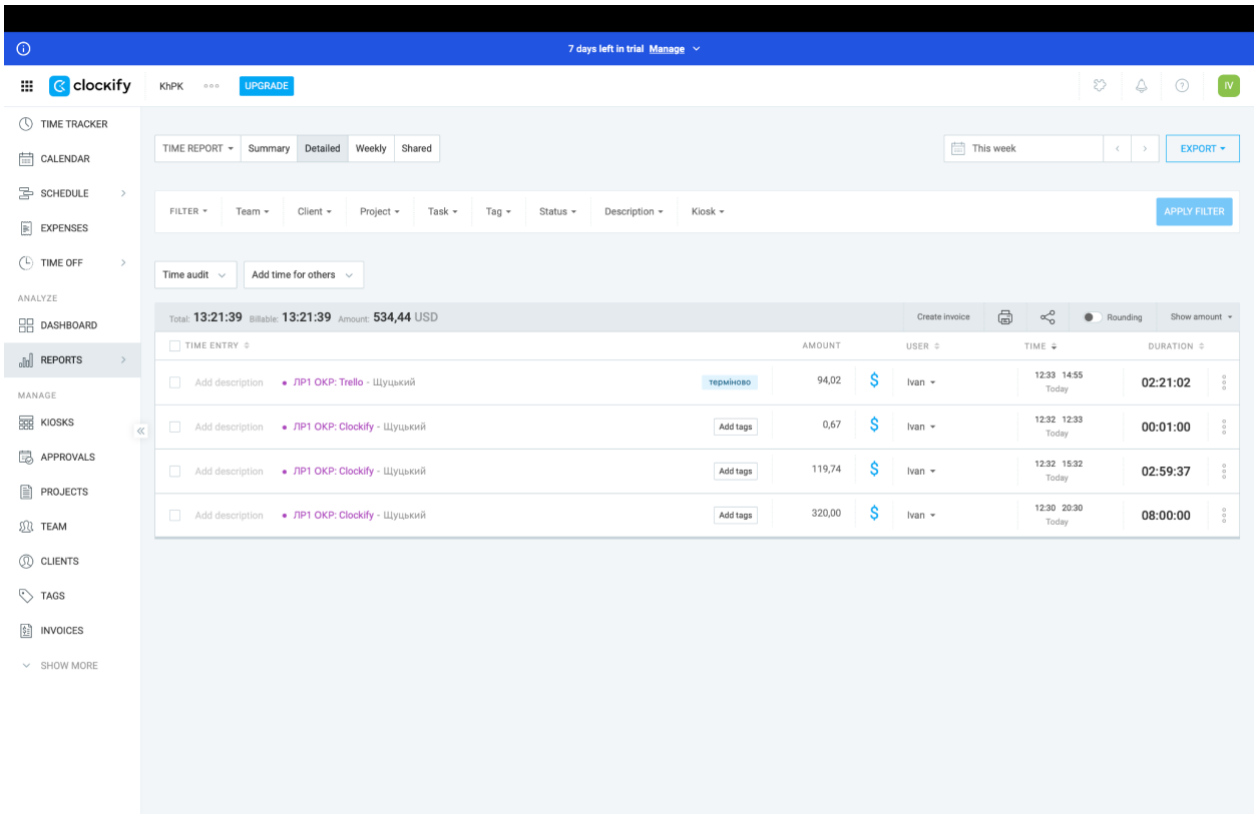
Панель моніторингу (Dashboard) з візуалізацією витраченого часу:



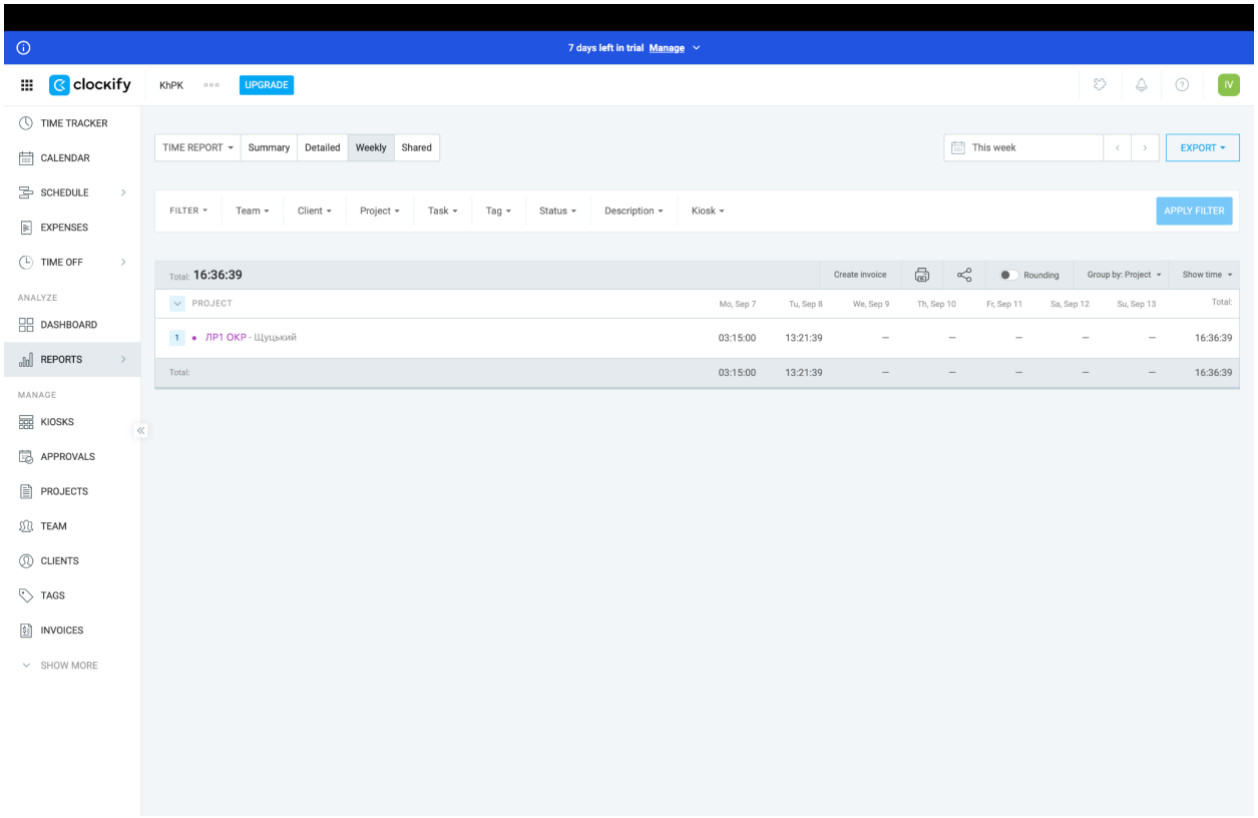
Підсумковий звіт (Summary Report) за проєктами:



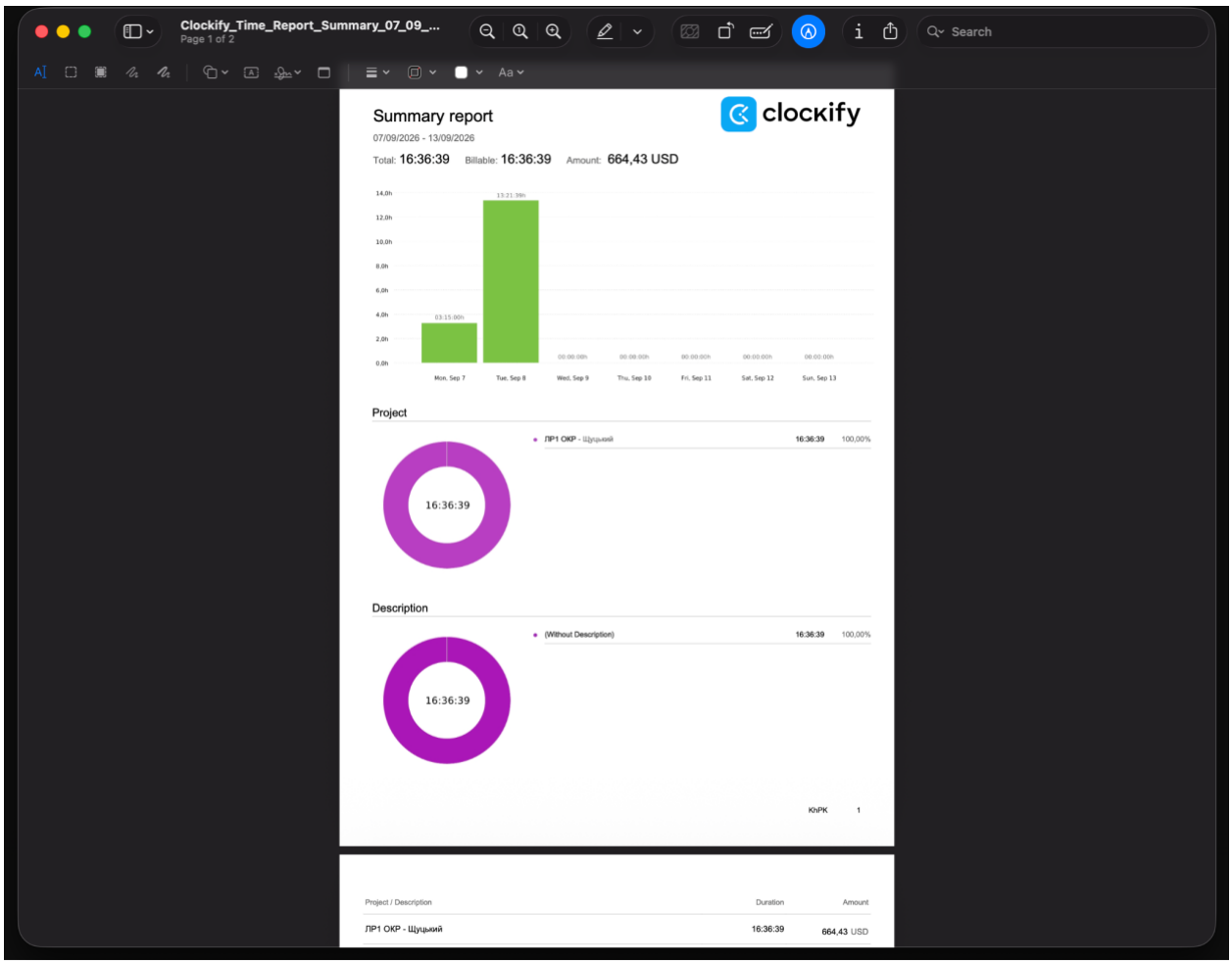
Деталізований звіт (Detailed Report) відстеженого часу:



Тижневий звіт (Weekly Report) витрат часу:



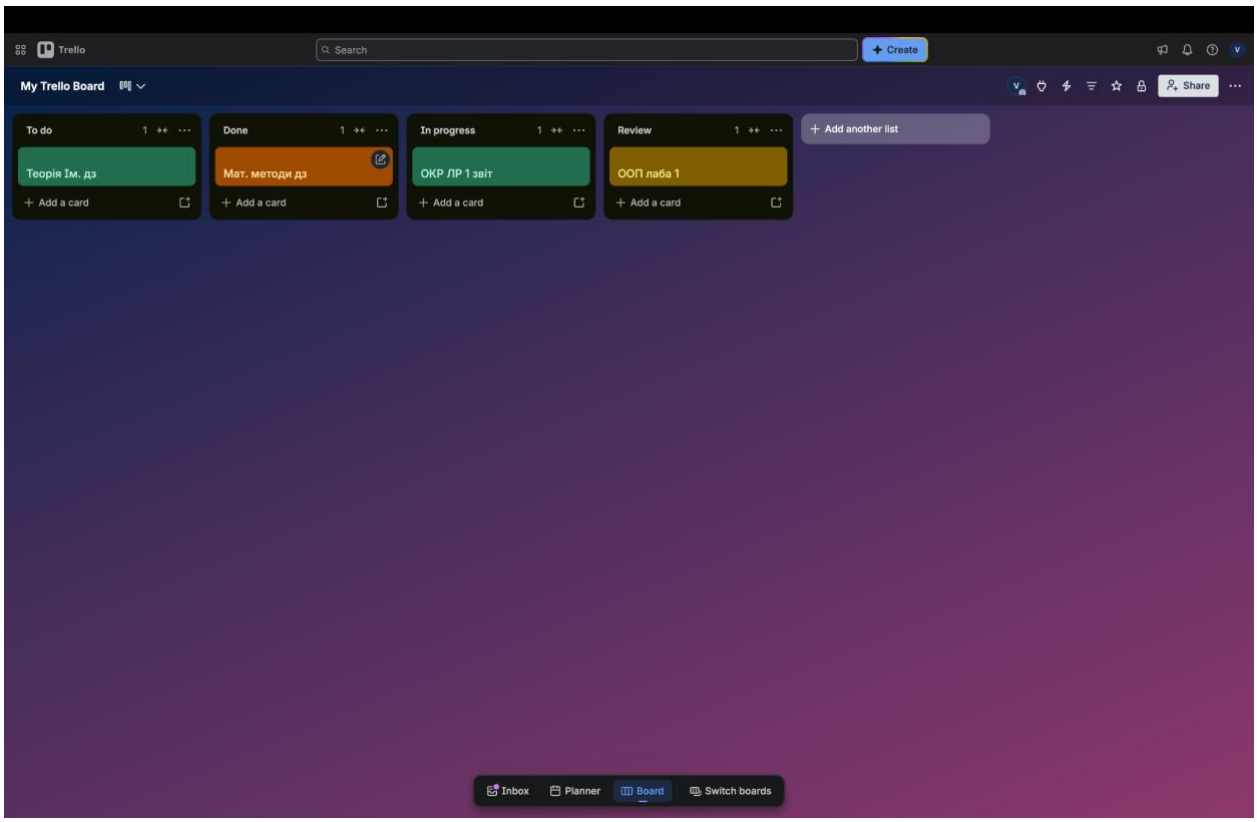
Експортований PDF-звіт з підсумковими даними:



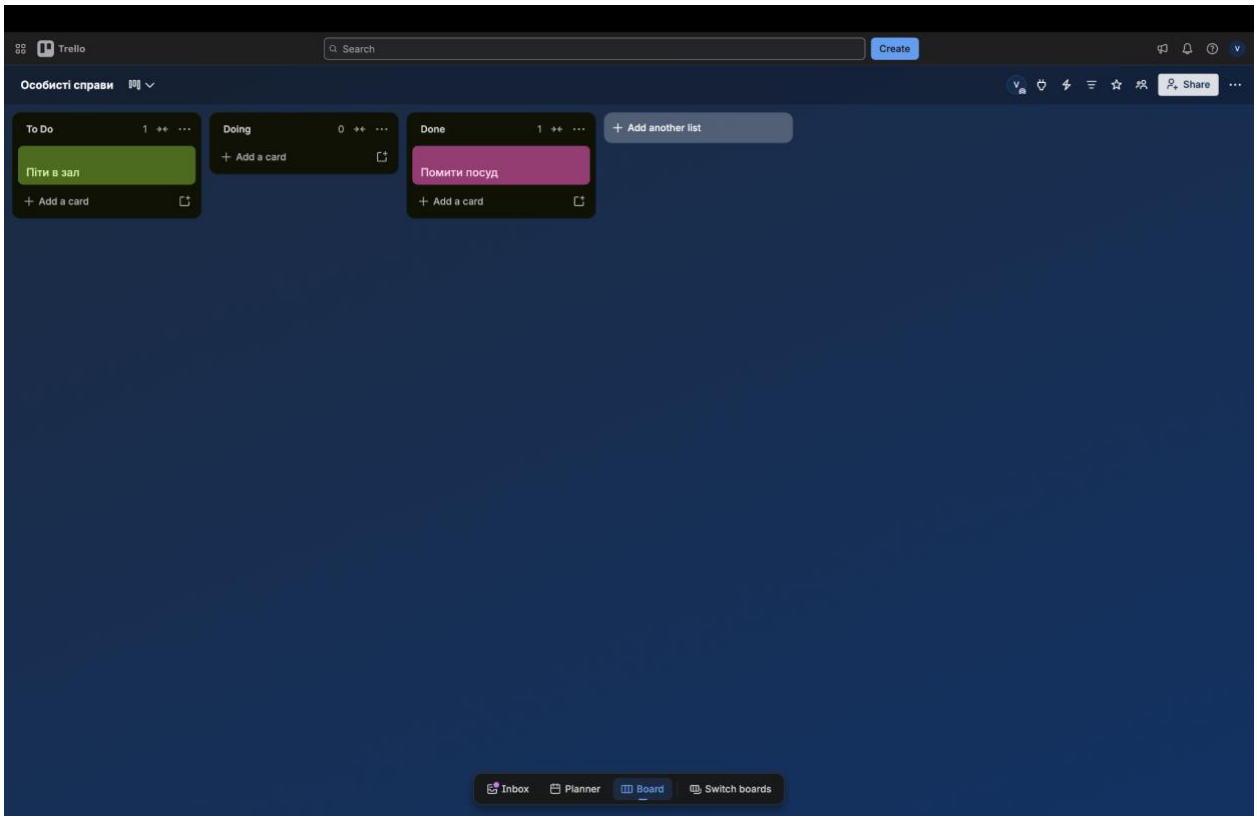
—--

Робота з Trello

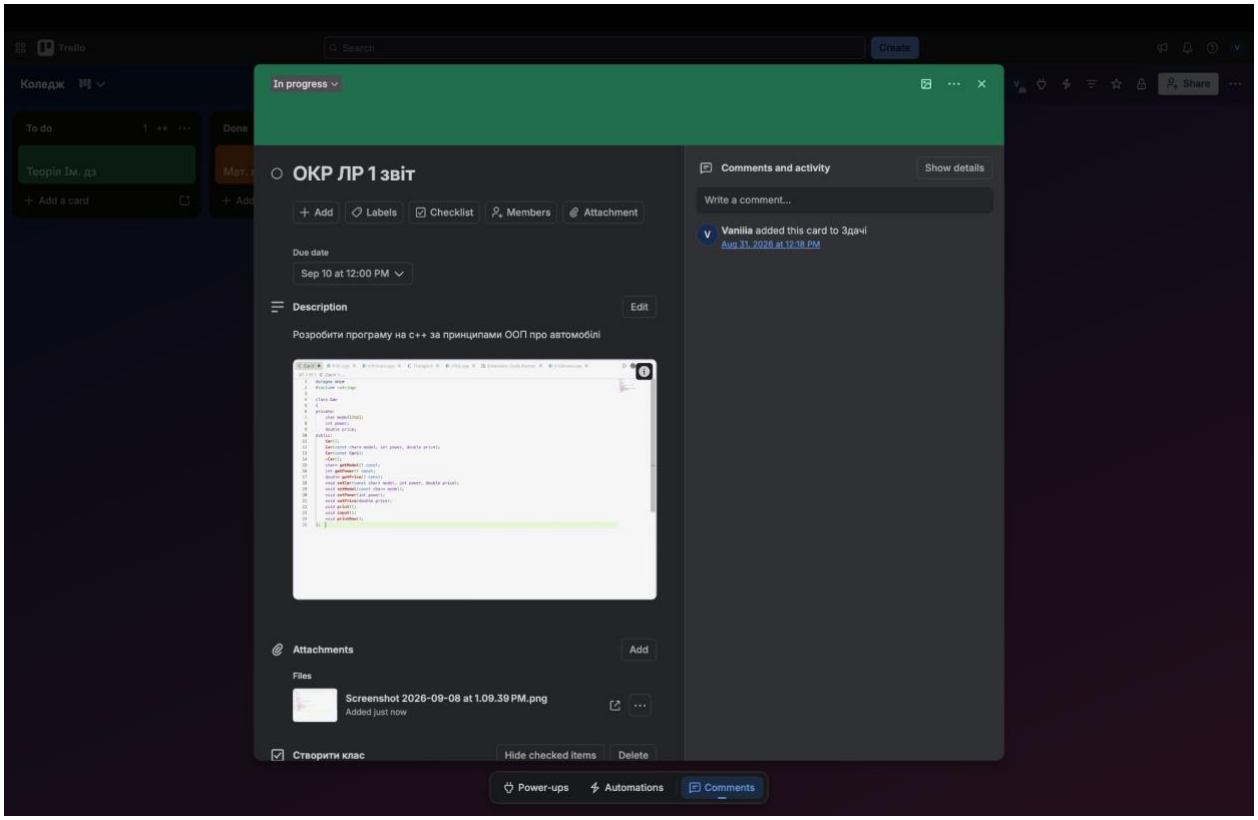
Головна дошка Trello зі списками завдань:



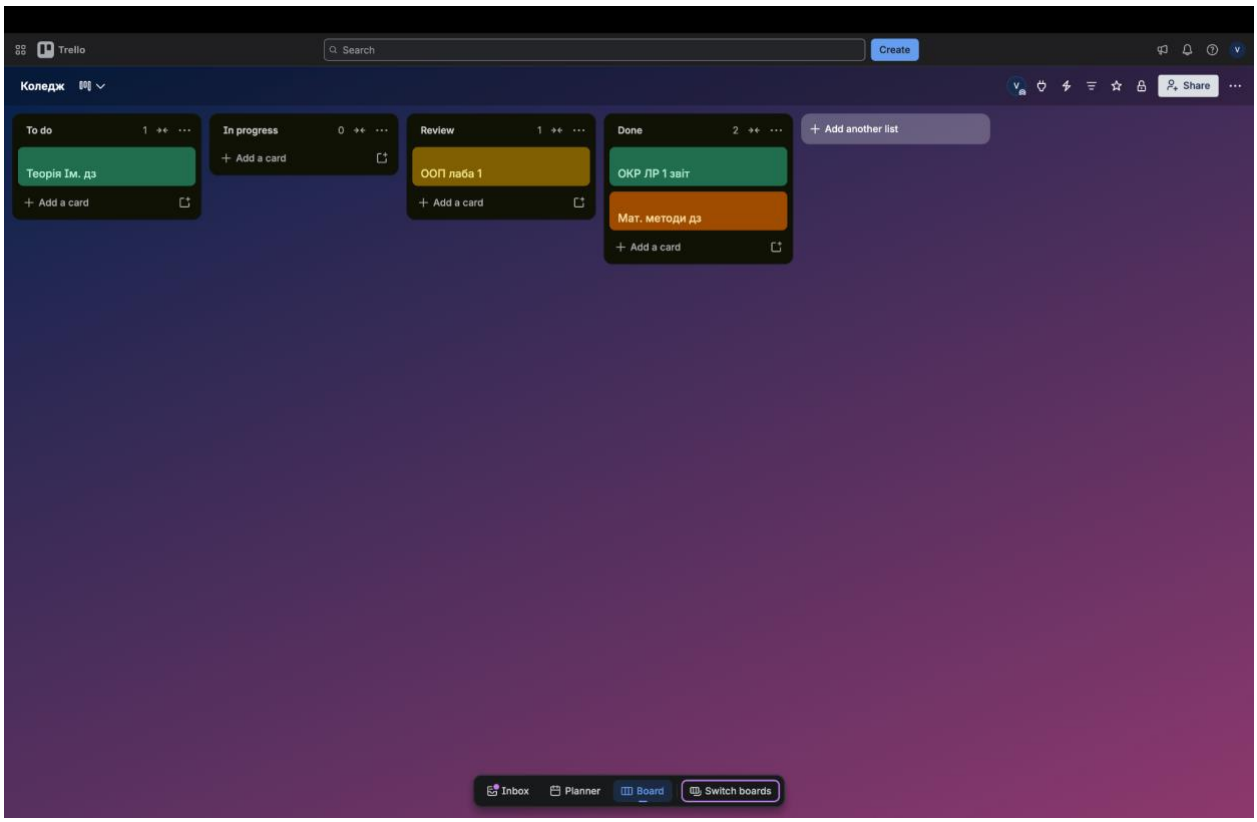
Перегляд особистих справ та списків задач у Trello:



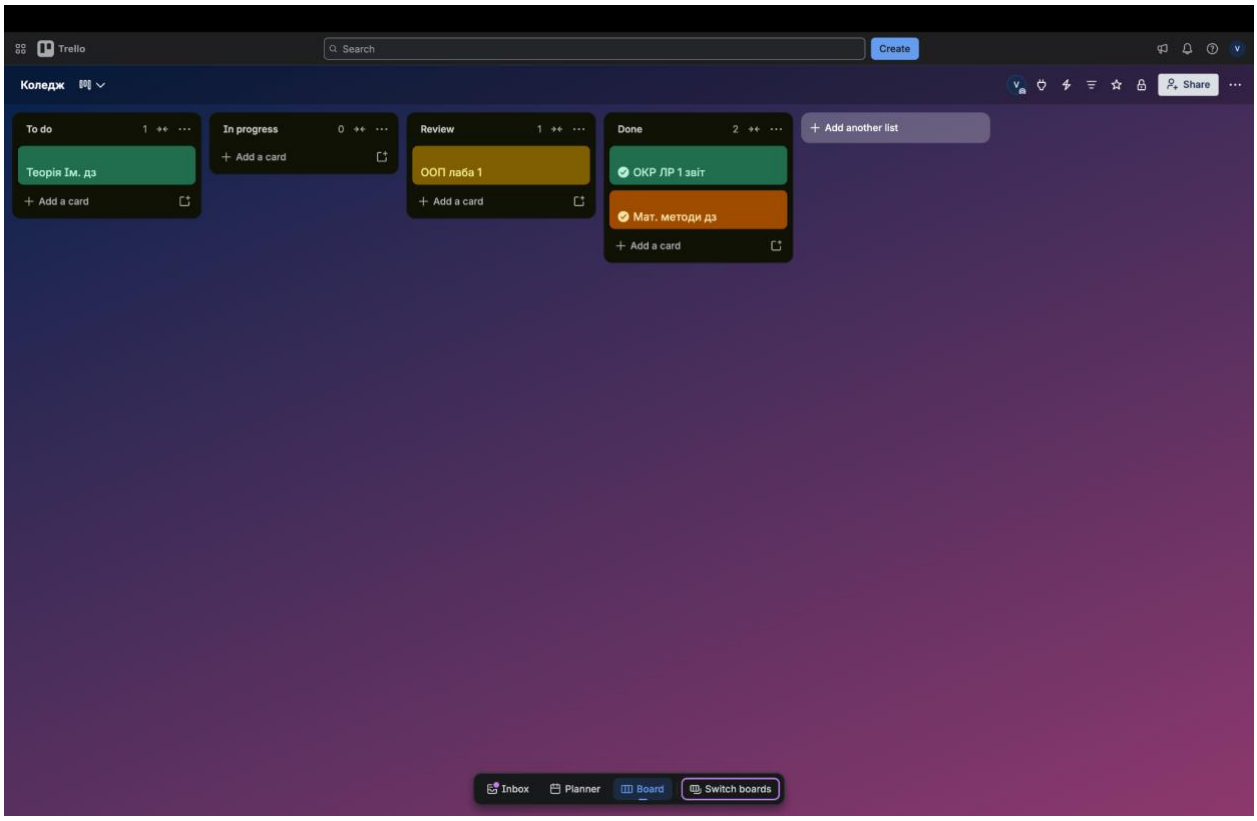
Деталі картки завдання з описом, термінами виконання та прикріпленими файлами:



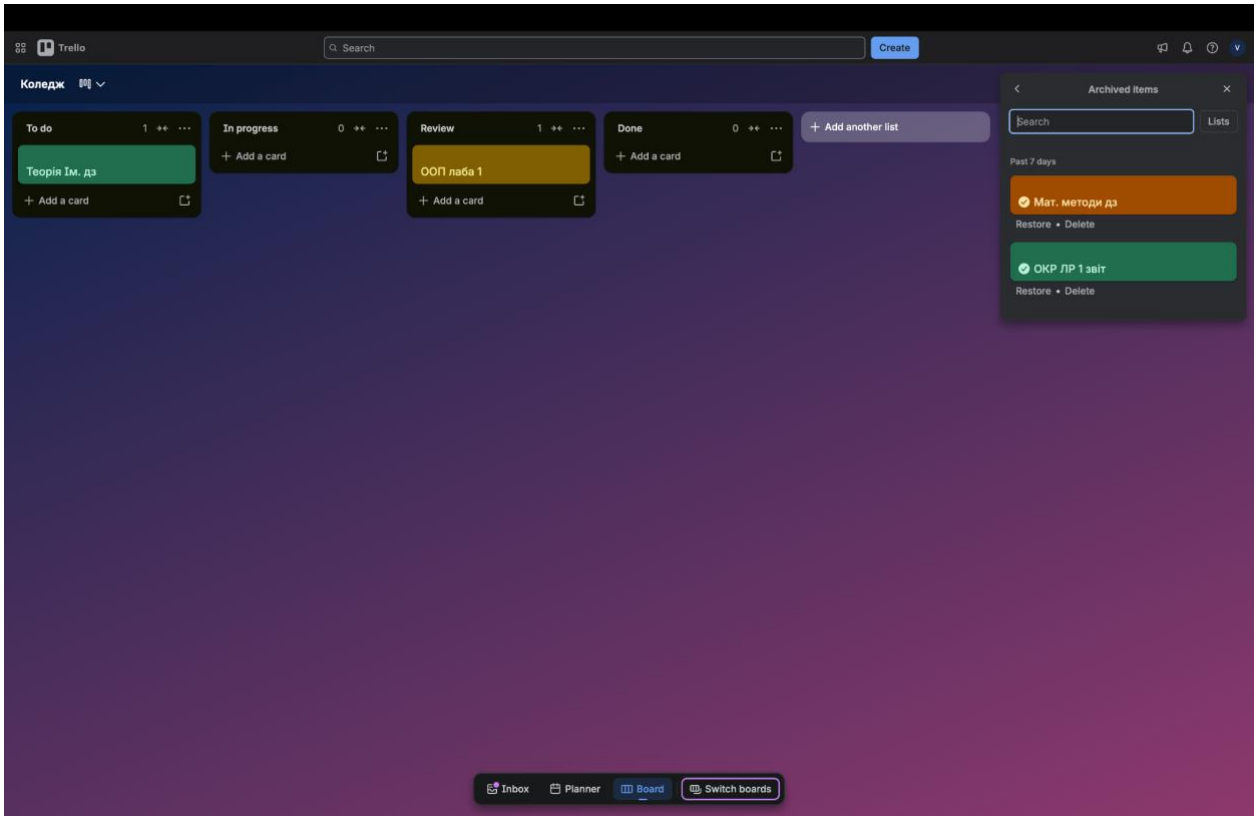
Переміщення виконаних завдань до колонки "Done":



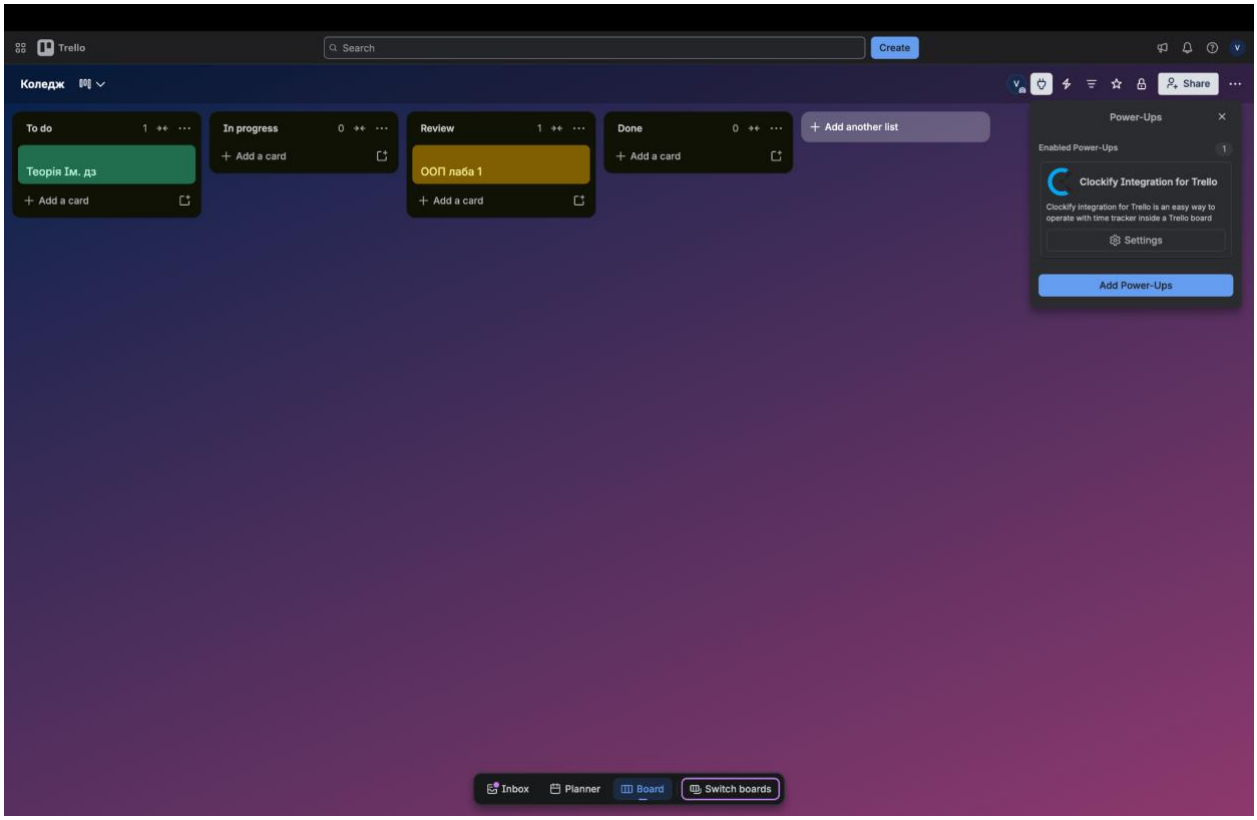
Відмічання завершених завдань у списку:



Перегляд архіву виконаних та архивованих карт завдань:



Налаштування інтеграції Power-Ups з сервісом Clockify у Trello:

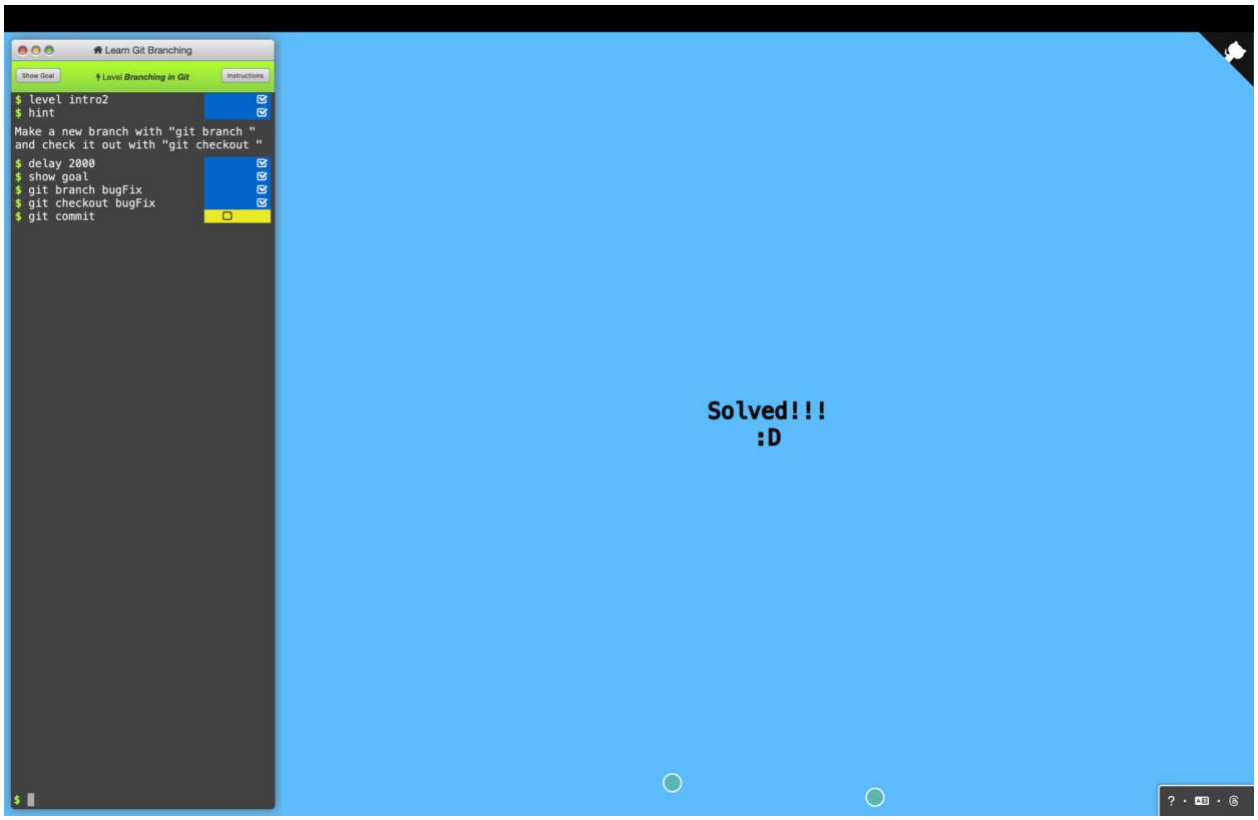


Робота з Learn Git Branching

1. Вступ до коммітів у Git (Introduction to Git Commits):



2. Створення та перемикання гілок (Branching in Git):



3. Злиття гілок (Merging in Git):



4. Перенесення коммітів (Rebase Introduction):



5. Відокремлений вказівник HEAD (Detach yo' HEAD):



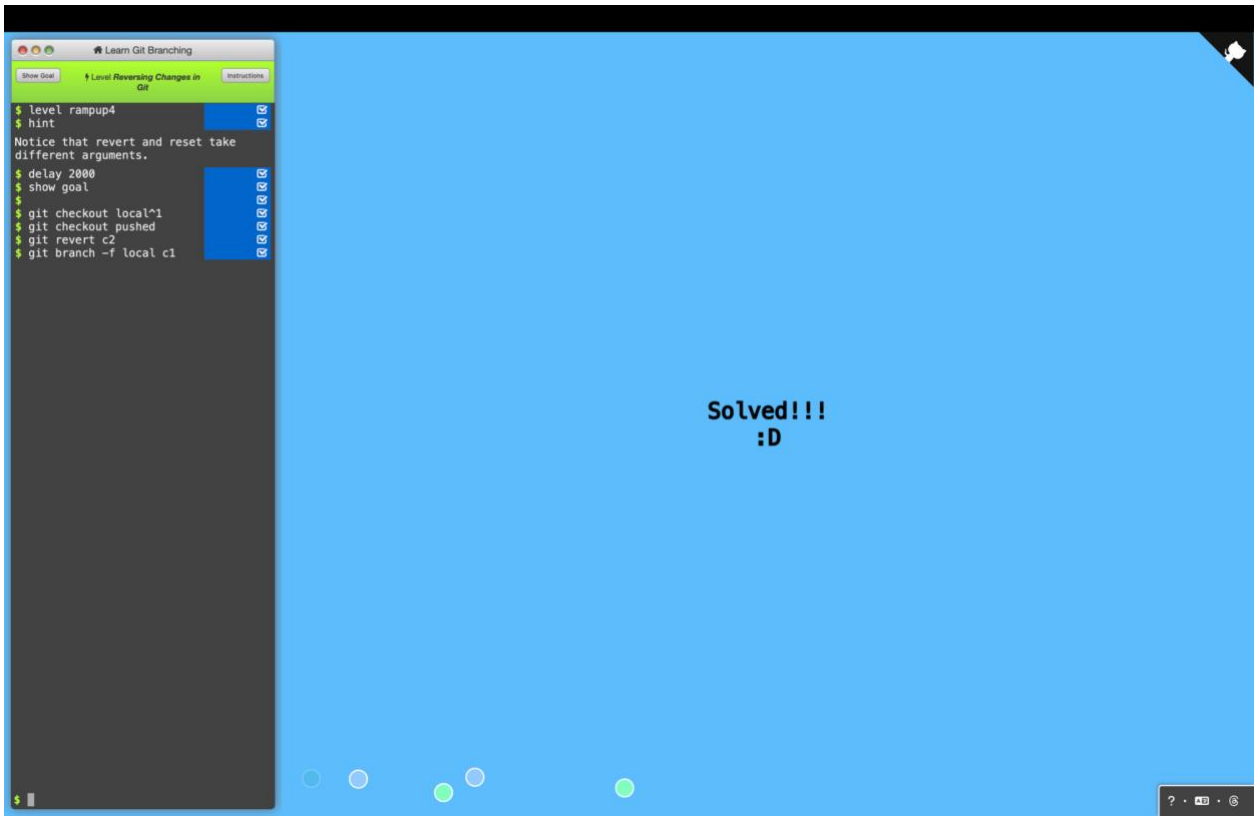
6. Відносні посилання у Git (Relative Refs (^)):



7. Відносні посилання ~ (Relative Refs #2 (~)):



8. Скасування змін у Git (Reversing Changes in Git):



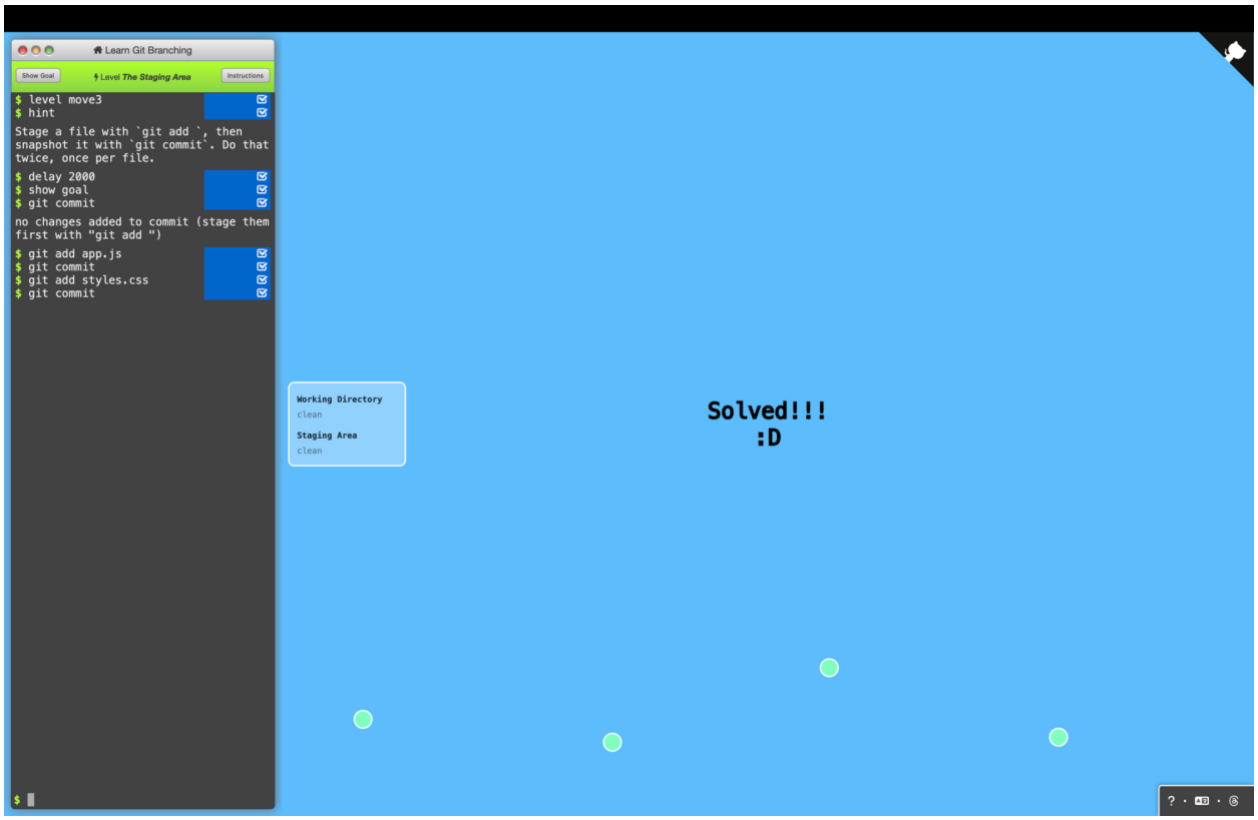
9. Копіювання коммітів за допомогою Cherry-pick (Cherry-pick Intro):



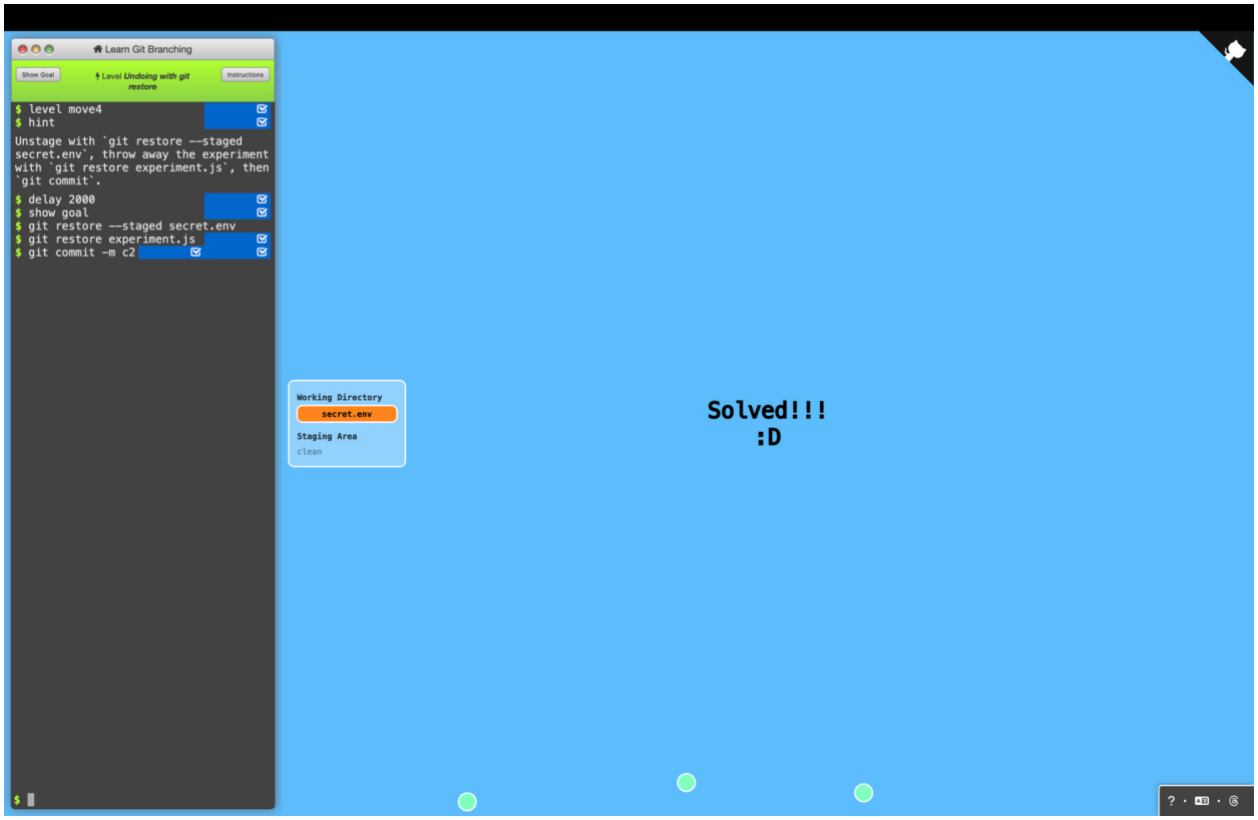
10. Інтерактивний Rebase (Interactive Rebase Intro):



11. Робота з індексом та Staging Area (The Staging Area):



12. Скасування змін за допомогою git restore (Undoing with git restore):



13. Перенесення одного комміту (Grabbing Just 1 Commit):



14. Маніпуляції з коммітами (Juggling Commits):



15. Маніпуляції з коммітами #2 (Juggling Commits #2):



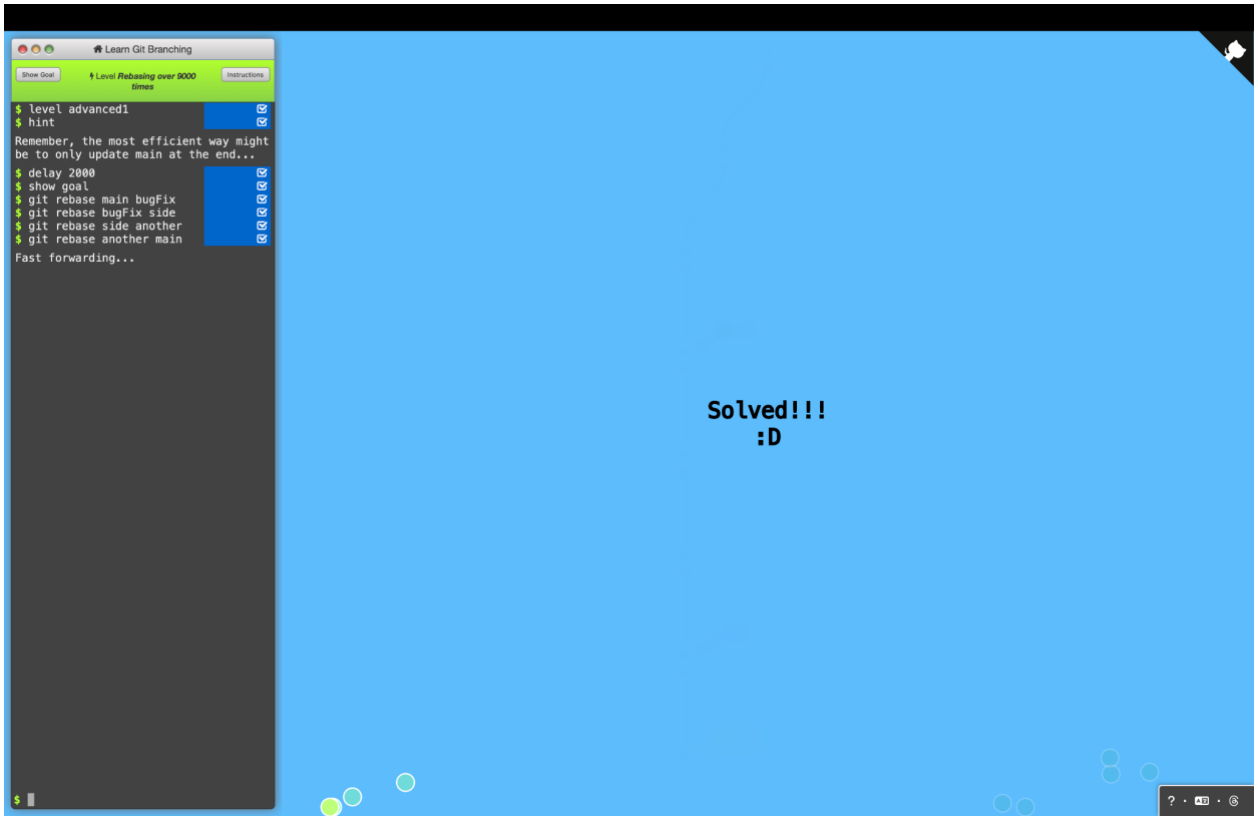
16. Теги у Git (Git Tags):



17. Використання команди Git Describe (Git Describe):



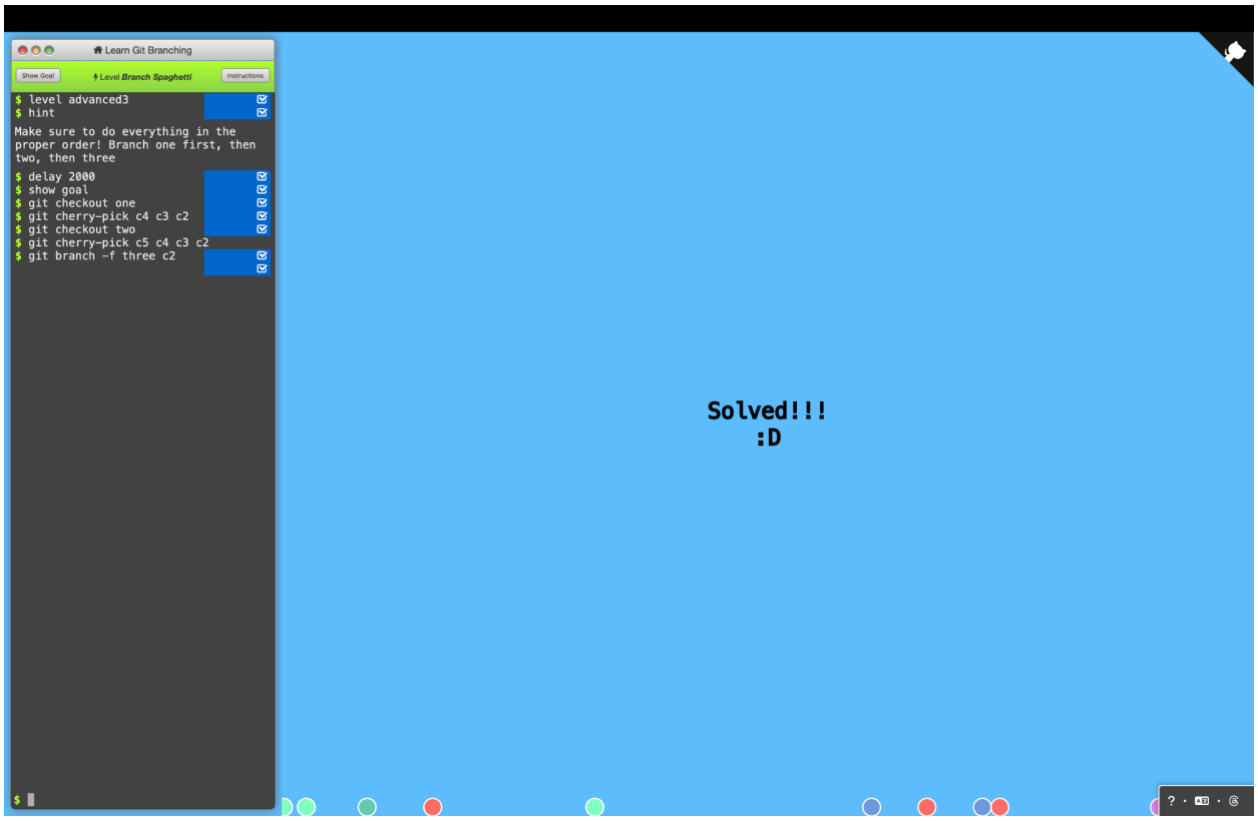
18. Багаторазовий Rebase (Rebasing over 9000 times):



19. Робота з декількома батьківськими коммітами (Multiple parents):



20. Складні маніпуляції з гілками (Branch Spaghetti):



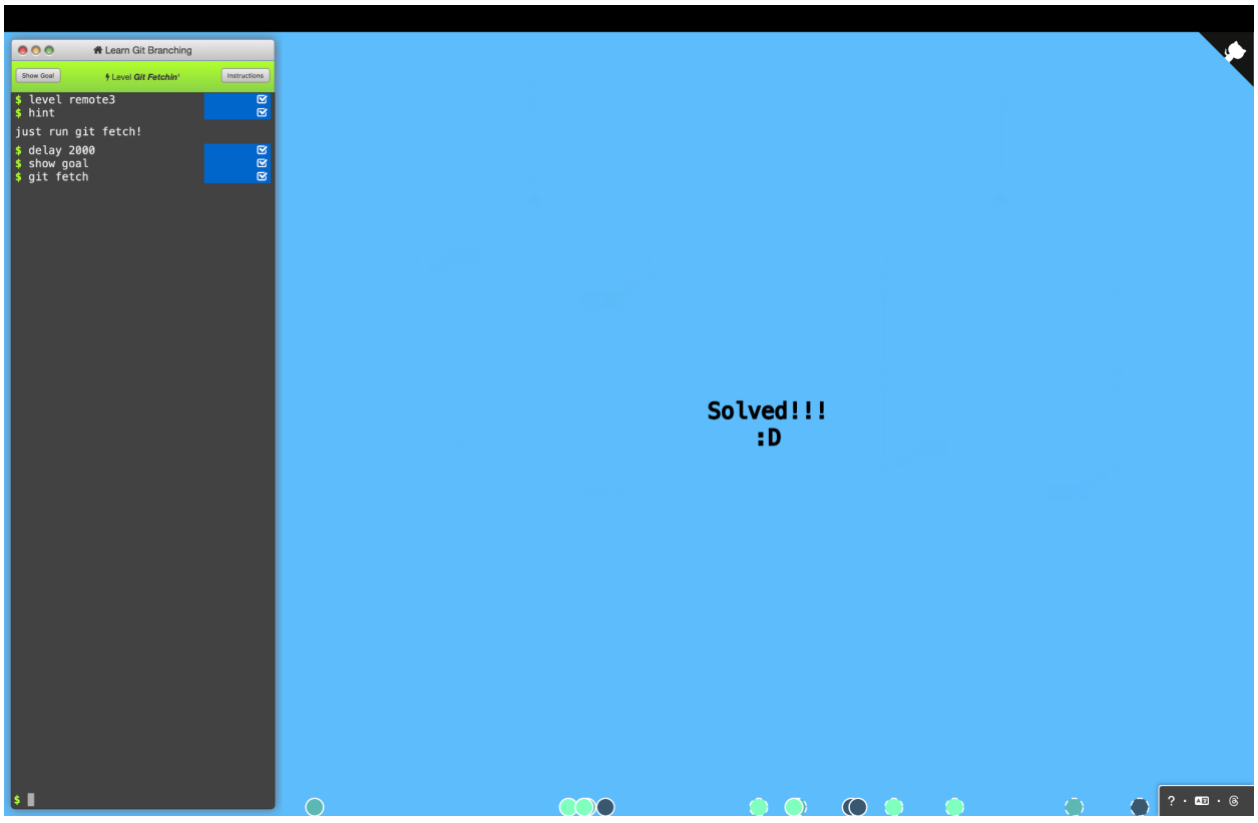
21. Клонування віддаленого репозиторію (Clone Intro):



22. Робота з віддаленими гілками (Remote Branches):



23. Отримання оновлень за допомогою Git Fetch (Git Fetchin'):



24. Отримання та злиття змін через Git Pull (Git Pullin'):



25. Симуляція командної роботи (Faking Teamwork):



26. Відправка змін у віддалений репозиторій (Git Pushin'):



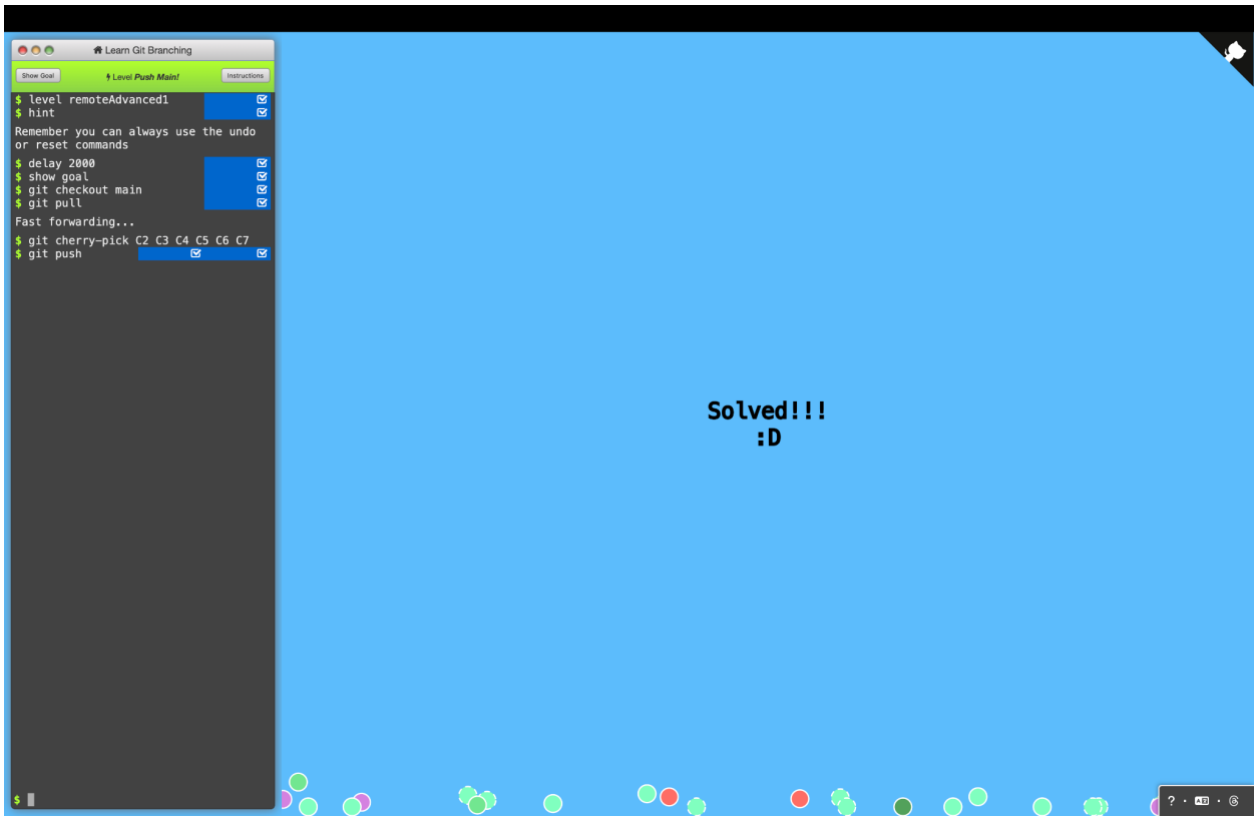
27. Розходження історії коммітів (Diverged History):



28. Заблокована основна гілка (Locked Main):



29. Пуш у головну гілку (Push Main!):



30. Злиття з віддаленими гілками (Merging with remotes):



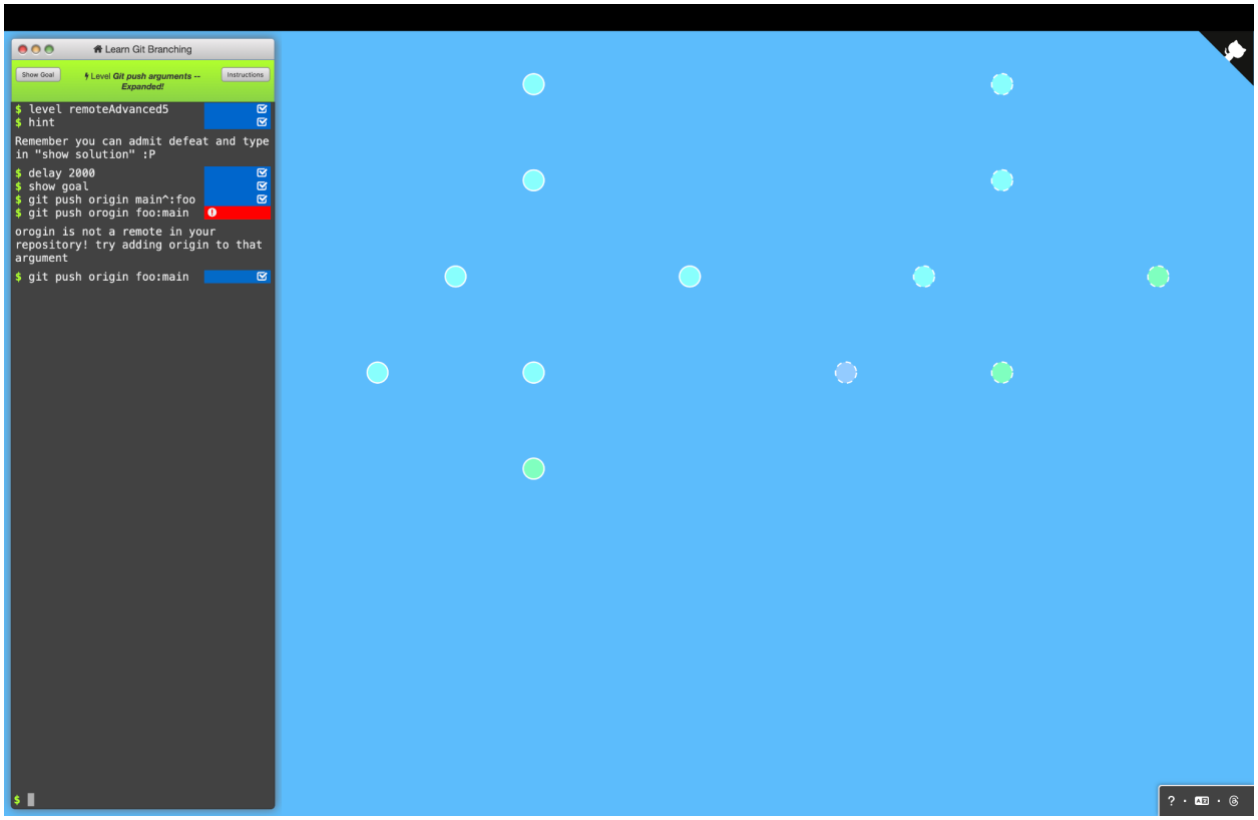
31. Налаштування відстеження віддалених гілок (Remote Tracking):



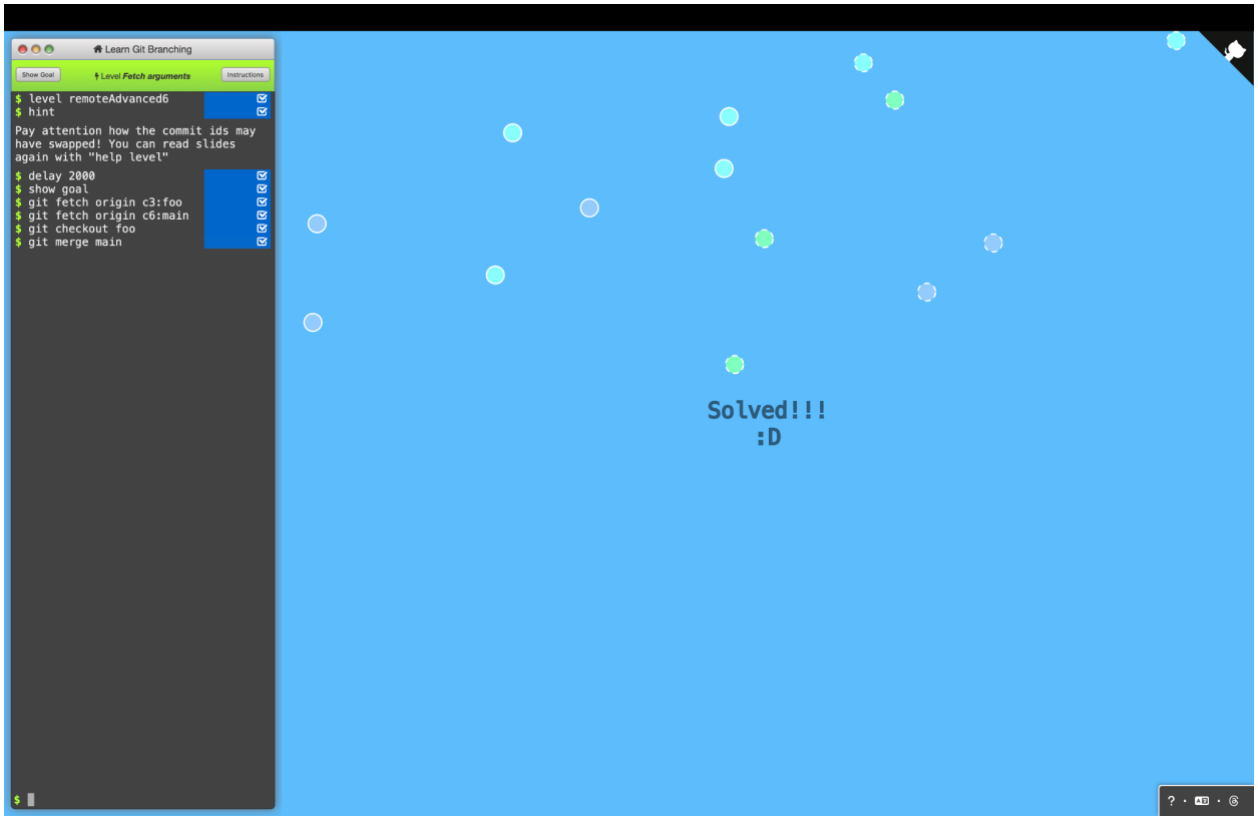
32. Аргументи команди Git Push (Git push arguments):



33. Розширені аргументи Git Push (Git push arguments -- Expanded!):



34. Аргументи команди Git Fetch (Fetch arguments):



35. Джерела та призначення для Fetch і Push (Source of nothing):



36. Аргументи команди Git Pull (Pull arguments):



Сертифікат про успішне проходження всіх рівнів Learn Git Branching:



Висновок

У ході виконання даної лабораторної роботи було опановано та практично застосовано індивідуальний робочий процес розробника програмного забезпечення із використанням сучасних інструментів управління проєктами, контролю часу та системи керування версіями.

Зокрема:

1. **Система керування версіями Git та GitHub**: Опановано налаштування конфігурації користувача, ініціалізацію репозиторію, створення та індексування коммітів, роботу з гілками (*dev*, *feature*), виконання злиття (*merge*) та перенесення (*rebase*), використання механізму тимчасового збереження змін (*stash*), а також підключення та синхронізацію з віддаленим репозиторієм на GitHub.

2. **Інтерактивне навчання Learn Git Branching**: Набуто практичних навичок візуального маніпулювання коммітами та гілками для глибшого розуміння внутрішнього влаштування Git.
3. **Контролер робочого часу Clockify**: Опановано ведення обліку часових витрат за завданнями, налаштування проєктів, створення тегів, керування командою, встановлення погодинних ставок та формування детальних і підсумкових звітів (зокрема в форматі PDF).
4. **Система управління проєктами Trello**: Організовано відстеження завдань за Scrum-підходом за допомогою дошок, списків (*To Do*, *In Progress*, *Review*, *Done*), карток завдань, опису та вкладень, а також налаштовано інтеграцію з сервісом Clockify (Power-Ups).

Завдяки інтегрованій взаємодії цих інструментів побудовано ефективний, прозорий та систематизований робочий процес для індивідуальної та командної розробки ПЗ.